

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

*



BÁO CÁO MÔN HỌC IT3160
WORD-LEVEL HANDWRITTEN TEXT RECOGNITION
NHẬN DẠNG CHỮ VIẾT TAY MỨC ĐỘ TỪ ĐƠN

Giảng viên phụ trách: PGS. TS. Ngô Văn Linh

Mã lớp: IT3160

Danh sách sinh viên:

STT	Họ tên	Mã sinh viên	Lớp
1	Nguyễn Trí Hiếu	202416203	CTTN KHMT K69
2	Đỗ Hải Đăng	202400035	CTTN KHMT K69
3	Đinh Thái Sơn	202416746	CTTN KHMT K69

Hà Nội, tháng 6 năm 2026

TÓM TẮT BÁO CÁO

Báo cáo trình bày quá trình xây dựng hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ sử dụng PyTorch. Hệ thống nhận đầu vào là ảnh chứa một từ tiếng Anh viết tay và sinh ra chuỗi ký tự tương ứng. Dự án được thực hiện trên bộ dữ liệu IAM Handwriting Dataset ở mức word-level với tổng cộng 96,835 mẫu sau khi làm sạch.

Trong phạm vi bài tập lớn, nhóm xây dựng và so sánh hai kiến trúc mô hình. Mô hình thứ nhất là CTC Baseline, kết hợp ResNet18, BiLSTM và CTC Loss. Mô hình thứ hai là Attention Model, kết hợp ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder.

Kết quả thực nghiệm trên tập kiểm thử IAM cho thấy CTC Baseline đạt 80.75% Word Accuracy và 7.27% Character Error Rate. Attention Model đạt 83.09% Word Accuracy và 7.06% Character Error Rate. Kết quả này cho thấy mô hình Attention có cải thiện so với baseline CTC trên cùng điều kiện đánh giá, tuy nhiên khả năng tổng quát hóa trên ảnh chữ viết tay thực tế vẫn còn là hạn chế cần tiếp tục cải thiện.

Mục lục

Tóm tắt báo cáo	i
1 Giới thiệu đề tài	1
1.1 Bối cảnh bài toán	1
1.2 Lý do chọn đề tài	1
1.3 Mục tiêu của dự án	1
1.4 Phạm vi thực hiện	2
1.5 Định hướng tiếp cận	2
1.6 Ý nghĩa của dự án	3
1.7 Cấu trúc báo cáo	3
2 Cơ sở lý thuyết	4
2.1 Tổng quan về bài toán nhận dạng chữ viết tay	4
2.2 Biểu diễn bài toán nhận dạng từ viết tay	5
2.3 Convolutional Neural Network	5
2.4 Mạng hồi quy và BiLSTM	6
2.5 Connectionist Temporal Classification	6
2.6 Cơ chế Attention	7
2.7 Kiến trúc Encoder–Decoder	8
2.8 Teacher Forcing	9
2.9 Các phương pháp giải mã	9
2.10 Các chỉ số đánh giá	10
2.11 Tổng kết chương	10
3 Dữ liệu và tiền xử lý	11
3.1 Giới thiệu bộ dữ liệu IAM	11
3.2 Đặc điểm dữ liệu	11
3.3 Cấu trúc dữ liệu đầu vào	12
3.4 Làm sạch nhãn	12
3.5 Xây dựng bộ từ vựng ký tự	12
3.6 Chia tập dữ liệu	13
3.7 Tiền xử lý ảnh	14
3.8 Kích thước ảnh đầu vào	14
3.9 Chuẩn hóa ảnh	15
3.10 Tăng cường dữ liệu	15
3.11 Xây dựng Dataset và DataLoader	16
3.12 Pipeline chuẩn bị dữ liệu	16
3.13 Một số hạn chế của dữ liệu	17
3.14 Tổng kết chương	17
4 Kiến trúc mô hình	18

4.1	Tổng quan kiến trúc hệ thống	18
4.2	CNN Encoder sử dụng ResNet18	19
4.3	Chuyển đổi Feature Map thành chuỗi đặc trưng	19
4.4	Mô hình CTC Baseline	20
4.5	Mô hình Attention	22
4.6	Giải mã trong mô hình Attention	24
4.7	So sánh hai mô hình	24
4.8	Vai trò của thư mục inference	24
4.9	Tổng kết chương	25
5	Cài đặt hệ thống	26
5.1	Môi trường phát triển	26
5.2	Phần cứng sử dụng	26
5.3	Cấu trúc thư mục dự án	27
5.4	Vai trò của các thư mục chính	28
5.5	Pipeline chuẩn bị dữ liệu	30
5.6	Pipeline huấn luyện	30
5.7	Chiến lược lưu checkpoint	32
5.8	Pipeline đánh giá	32
5.9	Pipeline dự đoán ảnh thực tế	33
5.10	Đầu ra của hệ thống	34
5.11	Quản lý phụ thuộc	34
5.12	Luồng chạy hệ thống	35
5.13	Tính mô-đun của hệ thống	35
5.14	Một số lưu ý khi triển khai	36
5.15	Tổng kết chương	36
6	Thực nghiệm và kết quả	37
6.1	Mục tiêu thực nghiệm	37
6.2	Thiết lập thực nghiệm	37
6.3	Các chỉ số đánh giá	38
6.4	Kết quả trên tập kiểm thử IAM	38
6.5	Phân tích kết quả	39
6.6	Vai trò của BiLSTM Context Encoder trong mô hình Attention	40
6.7	Một số ví dụ dự đoán	40
6.8	Trực quan hóa Attention	41
6.9	Kiểm thử trên ảnh thực tế	42
6.10	Phân tích lỗi	43
6.11	Đánh giá chung	43
6.12	Tổng kết chương	44
7	Kết luận và hướng phát triển	45
7.1	Kết luận	45
7.2	Những kết quả đạt được	45
7.3	Hạn chế của dự án	46
7.4	Bài học rút ra	46
7.5	Hướng phát triển	47
7.6	Kết luận chung	48
	Tài liệu tham khảo	49

Danh sách hình vẽ

2.1	Các mức độ của bài toán nhận dạng chữ viết tay	4
2.2	Minh họa ý tưởng căn chỉnh trong CTC	7
2.3	Minh họa cơ chế attention trong mô hình Encoder–Decoder	8
3.1	Một số ảnh mẫu từ IAM Handwriting Dataset ở mức word-level	11
3.2	Tỉ lệ phân chia dữ liệu train, validation và test	13
3.3	Pipeline tiền xử lý ảnh đầu vào	14
3.4	Ví dụ minh họa một số phép tăng cường dữ liệu	16
4.1	Pipeline tổng quan của hệ thống nhận dạng chữ viết tay	18
4.2	Minh họa quá trình chuyển feature map 2D thành chuỗi đặc trưng	20
4.3	Kiến trúc mô hình CTC Baseline	21
4.4	Kiến trúc mô hình Attention	22
5.1	Cấu trúc thư mục tổng quát của dự án	28
5.2	Luồng chuẩn bị dữ liệu trước khi huấn luyện	30
5.3	Pipeline suy diễn trên ảnh thực tế	34
5.4	Luồng chạy tổng quát của hệ thống	35
6.1	Biểu đồ so sánh kết quả CTC Baseline và Attention Model	39
6.2	Ví dụ trực quan một số dự đoán đúng và sai	41
6.3	Ví dụ attention map của mô hình Attention	42
6.4	Một số ví dụ kiểm thử trên ảnh chữ viết tay thực tế	43

Danh sách bảng

3.1	Phân chia bộ dữ liệu IAM sau tiền xử lý	13
3.2	Kích thước ảnh đầu vào của hai mô hình	15
4.1	So sánh kiến trúc CTC Baseline và Attention Model	24
5.1	Môi trường phát triển của dự án	26
5.2	Phần cứng sử dụng trong quá trình huấn luyện	26
6.1	Phân chia dữ liệu trong thực nghiệm	37
6.2	Tóm tắt thiết lập thực nghiệm	38
6.3	Kết quả so sánh hai mô hình trên IAM Test Set	39
6.4	Kết quả chi tiết của hai mô hình	39
6.5	Một số ví dụ dự đoán của hai mô hình	41

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh bài toán

Trong quá trình số hóa tài liệu, nhu cầu chuyển đổi nội dung từ ảnh sang văn bản có thể chỉnh sửa được ngày càng trở nên phổ biến. Các hệ thống nhận dạng ký tự quang học, thường được gọi là OCR, đã được ứng dụng rộng rãi trong việc xử lý văn bản in, biểu mẫu, hóa đơn, tài liệu lưu trữ và nhiều loại dữ liệu dạng ảnh khác. Tuy nhiên, đối với chữ viết tay, bài toán nhận dạng vẫn còn nhiều thách thức hơn so với chữ in do sự khác biệt lớn về nét chữ giữa từng người viết.

Chữ viết tay có tính biến thiên cao. Cùng một ký tự có thể được viết theo nhiều hình dạng khác nhau, phụ thuộc vào thói quen viết, độ nghiêng, kích thước chữ, khoảng cách giữa các ký tự, độ dày nét bút và chất lượng ảnh đầu vào. Ngoài ra, ảnh chữ viết tay có thể bị ảnh hưởng bởi nhiễu, nền giấy, độ sáng không đồng đều hoặc quá trình scan/chụp ảnh. Những yếu tố này khiến việc nhận dạng chữ viết tay trở thành một bài toán khó và có ý nghĩa thực tiễn trong lĩnh vực thị giác máy tính.

Trong phạm vi bài tập lớn này, nhóm tập trung vào bài toán nhận dạng chữ viết tay ở mức độ từ đơn lẻ. Đầu vào của hệ thống là một ảnh chứa duy nhất một từ tiếng Anh viết tay. Đầu ra là chuỗi ký tự tương ứng với nội dung trong ảnh. Đây là một mức bài toán phù hợp để nghiên cứu các phương pháp học sâu cho nhận dạng chuỗi ký tự, đồng thời vẫn giữ được phạm vi triển khai hợp lý trong khuôn khổ môn học.

1.2. Lý do chọn đề tài

Nhóm lựa chọn đề tài *Word-Level Handwritten Text Recognition* vì đây là bài toán có sự kết hợp giữa nhiều mảng kiến thức quan trọng trong học máy và học sâu, bao gồm xử lý ảnh, trích xuất đặc trưng, mô hình hóa chuỗi và giải mã chuỗi ký tự. Thông qua đề tài này, nhóm có thể vận dụng các kiến thức về mạng tích chập, mạng hồi quy, cơ chế attention và các chỉ số đánh giá thường dùng trong bài toán nhận dạng văn bản.

Bên cạnh đó, bài toán nhận dạng chữ viết tay không chỉ dừng lại ở việc phân loại một ảnh vào một nhãn cố định, mà cần sinh ra một chuỗi ký tự có độ dài thay đổi. Điều này làm cho bài toán trở nên thú vị hơn so với các bài toán phân loại ảnh thông thường. Mô hình cần học cách biến đổi ảnh đầu vào thành một chuỗi đầu ra, trong khi không có thông tin căn chỉnh trực tiếp giữa từng vùng ảnh và từng ký tự.

Dự án sử dụng IAM Handwriting Dataset ở mức word-level, một bộ dữ liệu phổ biến trong các nghiên cứu về nhận dạng chữ viết tay. Việc sử dụng một bộ dữ liệu chuẩn giúp quá trình huấn luyện và đánh giá có cơ sở rõ ràng hơn. Đồng thời, nhóm có thể so sánh hai hướng tiếp cận khác nhau trên cùng một điều kiện dữ liệu: mô hình sử dụng CTC Loss và mô hình Encoder-Decoder sử dụng cơ chế Attention.

1.3. Mục tiêu của dự án

Mục tiêu chính của dự án là xây dựng một hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ chạy trên môi trường local bằng PyTorch. Hệ thống cần có khả năng nhận ảnh đầu vào, tiền xử lý ảnh, đưa ảnh qua mô

hình học sâu và trả về chuỗi ký tự dự đoán.

Các mục tiêu cụ thể của dự án bao gồm:

- Tìm hiểu bài toán nhận dạng chữ viết tay mức độ từ đơn lẻ.
- Chuẩn bị và tiền xử lý bộ dữ liệu IAM Handwriting Dataset ở mức word-level.
- Xây dựng pipeline dữ liệu phục vụ huấn luyện, kiểm định và kiểm thử mô hình.
- Cài đặt mô hình CTC Baseline sử dụng ResNet18, BiLSTM và CTC Loss.
- Cài đặt mô hình Attention sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder.
- Huấn luyện, đánh giá và so sánh hai mô hình trên cùng tập dữ liệu.
- Sử dụng các chỉ số Word Accuracy, Character Error Rate và Normalized Edit Distance để đánh giá kết quả.
- Xây dựng chức năng dự đoán cho ảnh đơn lẻ hoặc toàn bộ thư mục ảnh.
- Trực quan hóa attention map đối với mô hình Attention để hỗ trợ phân tích kết quả.

1.4. Phạm vi thực hiện

Trong phạm vi bài tập lớn, dự án chỉ tập trung vào nhận dạng chữ viết tay ở mức độ từ đơn lẻ. Mỗi ảnh đầu vào được giả định chỉ chứa một từ tiếng Anh. Hệ thống không xử lý nhận dạng toàn bộ dòng văn bản, đoạn văn hoặc tài liệu hoàn chỉnh. Các bước như phát hiện vùng chữ, tách dòng, tách từ trong một trang tài liệu không nằm trong phạm vi triển khai của dự án.

Dự án được thực hiện hoàn toàn trên môi trường local và không sử dụng các dịch vụ OCR API bên thứ ba. Các mô hình được xây dựng, huấn luyện và đánh giá bằng PyTorch. Dữ liệu chính được sử dụng là IAM Handwriting Dataset sau khi đã được làm sạch và chia thành các tập train, validation và test.

Về mặt mô hình, nhóm tập trung so sánh hai hướng tiếp cận chính. Hướng thứ nhất là mô hình CTC Baseline, phù hợp với các bài toán chuỗi không có thông tin căn chỉnh giữa đầu vào và đầu ra. Hướng thứ hai là mô hình Attention theo kiến trúc Encoder–Decoder, cho phép mô hình học cơ chế căn chỉnh mềm giữa đặc trưng ảnh và chuỗi ký tự đầu ra.

1.5. Định hướng tiếp cận

Để giải quyết bài toán, nhóm xây dựng hệ thống theo pipeline gồm các bước chính: tiền xử lý dữ liệu, trích xuất đặc trưng ảnh, mô hình hóa chuỗi, giải mã chuỗi ký tự và đánh giá kết quả.

Đầu tiên, ảnh chữ viết tay được đọc, chuẩn hóa kích thước và chuyển về dạng tensor phù hợp với đầu vào của mô hình. Sau đó, ResNet18 được sử dụng như một bộ trích xuất đặc trưng ảnh. Feature map thu được từ CNN được chuyển đổi thành chuỗi đặc trưng theo chiều ngang của ảnh, vì chiều ngang thường tương ứng với thứ tự xuất hiện của các ký tự trong từ.

Với mô hình CTC Baseline, chuỗi đặc trưng được đưa qua BiLSTM và lớp chiếu tuyến tính để dự đoán phân phối ký tự tại từng bước thời gian. CTC Loss được sử dụng để học căn chỉnh giữa chuỗi đặc trưng và chuỗi nhãn mà không cần biết vị trí chính xác của từng ký tự trong ảnh.

Với mô hình Attention, chuỗi đặc trưng sau CNN được xử lý thêm bằng BiLSTM Context Encoder. Sau đó, Bahdanau Attention được sử dụng để tính vùng đặc trưng cần tập trung tại mỗi bước giải mã. GRU Decoder sinh chuỗi ký tự đầu ra từng bước cho đến khi gặp ký hiệu kết thúc hoặc đạt độ dài tối đa.

1.6. Ý nghĩa của dự án

Dự án giúp nhóm hiểu rõ hơn về cách xây dựng một hệ thống học sâu cho bài toán nhận dạng chuỗi từ dữ liệu ảnh. Thay vì chỉ sử dụng một mô hình có sẵn, nhóm triển khai đầy đủ các thành phần từ xử lý dữ liệu, thiết kế kiến trúc, huấn luyện, đánh giá đến suy diễn trên ảnh thực tế.

Việc so sánh CTC Baseline và Attention Model cũng giúp làm rõ sự khác biệt giữa hai hướng tiếp cận phổ biến trong bài toán nhận dạng văn bản. CTC có ưu điểm là đơn giản, ổn định và không cần decoder phức tạp. Trong khi đó, Attention Model linh hoạt hơn, có khả năng học căn chỉnh động và có thể trực quan hóa vùng ảnh mà mô hình tập trung khi dự đoán từng ký tự.

Bên cạnh kết quả định lượng trên tập kiểm thử, dự án cũng cho thấy một vấn đề thực tế quan trọng: mô hình có thể đạt kết quả tốt trên bộ dữ liệu chuẩn nhưng vẫn suy giảm hiệu năng khi áp dụng trên ảnh chữ viết tay ngoài môi trường chuẩn. Điều này giúp nhóm có cái nhìn thận trọng hơn về khả năng tổng quát hóa của mô hình học sâu.

1.7. Cấu trúc báo cáo

Báo cáo được tổ chức thành các chương như sau:

- **Chương 1** giới thiệu bối cảnh, lý do chọn đề tài, mục tiêu, phạm vi và định hướng tiếp cận của dự án.
- **Chương 2** trình bày các cơ sở lý thuyết liên quan đến nhận dạng chữ viết tay, CNN, BiLSTM, CTC, Attention và các chỉ số đánh giá.
- **Chương 3** mô tả bộ dữ liệu IAM, quá trình làm sạch nhãn, chia tập dữ liệu, tiền xử lý ảnh và xây dựng DataLoader.
- **Chương 4** trình bày chi tiết kiến trúc hai mô hình được triển khai: CTC Baseline và Attention Model.
- **Chương 5** mô tả cách cài đặt hệ thống, cấu trúc thư mục dự án, pipeline huấn luyện, đánh giá và dự đoán.
- **Chương 6** trình bày thiết lập thực nghiệm, kết quả đánh giá, phân tích kết quả và nhận xét về khả năng tổng quát hóa.
- **Chương 7** tổng kết dự án, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về bài toán nhận dạng chữ viết tay

Nhận dạng chữ viết tay, hay Handwritten Text Recognition (HTR), là bài toán chuyển đổi ảnh chứa chữ viết tay thành chuỗi văn bản tương ứng. Đây là một bài toán thuộc lĩnh vực thị giác máy tính, đồng thời có liên hệ chặt chẽ với xử lý chuỗi trong học sâu.

Khác với nhận dạng chữ in, chữ viết tay có độ biến thiên rất lớn giữa các cá nhân. Cùng một ký tự có thể được viết với nhiều hình dạng khác nhau, phụ thuộc vào thói quen viết, độ nghiêng, tốc độ viết, loại bút, khoảng cách giữa các ký tự và chất lượng ảnh đầu vào. Vì vậy, mô hình nhận dạng chữ viết tay cần học được cả đặc trưng hình ảnh cục bộ và mối quan hệ tuần tự giữa các ký tự trong từ.

Tùy theo phạm vi đầu vào, bài toán HTR có thể được chia thành nhiều mức độ:

- **Character-level recognition:** nhận dạng từng ký tự riêng lẻ.
- **Word-level recognition:** nhận dạng một ảnh chứa một từ đơn lẻ.
- **Line-level recognition:** nhận dạng một dòng văn bản viết tay.
- **Paragraph-level hoặc document-level recognition:** nhận dạng đoạn văn hoặc toàn bộ tài liệu.

Trong phạm vi dự án này, nhóm tập trung vào bài toán word-level HTR. Mỗi ảnh đầu vào chỉ chứa một từ tiếng Anh viết tay, và đầu ra là chuỗi ký tự tương ứng với từ đó.

HTR Task Levels (Mức độ bài toán nhận dạng chữ viết tay)



Hình 2.1: Các mức độ của bài toán nhận dạng chữ viết tay

2.2. Biểu diễn bài toán nhận dạng từ viết tay

Về mặt hình thức, bài toán nhận dạng chữ viết tay mức độ từ đơn có thể được mô tả như sau. Cho một ảnh đầu vào X chứa một từ viết tay, mô hình cần dự đoán chuỗi ký tự:

$$Y = (y_1, y_2, \dots, y_L)$$

Trong đó L là độ dài của từ, và mỗi y_i là một ký tự thuộc bộ từ vựng đã định nghĩa. Khó khăn chính của bài toán nằm ở việc độ dài chuỗi đầu ra không cố định, trong khi ảnh đầu vào có kích thước cố định sau tiền xử lý.

Ngoài ra, hệ thống không có thông tin căn chỉnh trực tiếp giữa từng vùng ảnh và từng ký tự trong nhãn. Nói cách khác, ta biết ảnh tương ứng với từ nào, nhưng không biết ký tự thứ nhất, thứ hai, hay thứ ba nằm chính xác tại vị trí nào trong ảnh. Vì vậy, bài toán cần các phương pháp có khả năng học căn chỉnh ngầm hoặc căn chỉnh động giữa đặc trưng ảnh và chuỗi ký tự.

Trong dự án, nhóm triển khai hai hướng tiếp cận:

- **CTC-based approach:** sử dụng CTC Loss để học căn chỉnh ngầm giữa chuỗi đặc trưng ảnh và chuỗi ký tự.
- **Attention-based approach:** sử dụng cơ chế attention để học căn chỉnh mềm giữa các vị trí đặc trưng ảnh và từng bước giải mã ký tự.

2.3. Convolutional Neural Network

Convolutional Neural Network (CNN) là một kiến trúc mạng nơ-ron thường được sử dụng trong các bài toán xử lý ảnh. CNN có khả năng học các đặc trưng không gian từ ảnh thông qua các lớp tích chập. Ở các tầng đầu, mạng thường học các đặc trưng đơn giản như cạnh, đường nét, góc và kết cấu. Ở các tầng sâu hơn, mạng có thể học các đặc trưng trừu tượng hơn như hình dạng ký tự hoặc cấu trúc của nét chữ.

Trong bài toán nhận dạng chữ viết tay, CNN được sử dụng để biến ảnh đầu vào thành feature map. Feature map này chứa thông tin đặc trưng của ảnh, sau đó được chuyển thành chuỗi đặc trưng theo chiều ngang để xử lý bằng các mô hình tuần tự.

Dự án sử dụng ResNet18 làm CNN Encoder. ResNet18 là một kiến trúc CNN có sử dụng residual connection, giúp việc huấn luyện mạng sâu ổn định hơn. Thay vì sử dụng ResNet18 như một mô hình phân loại ảnh hoàn chỉnh, dự án chỉ sử dụng phần trích xuất đặc trưng của mạng.

Với ảnh đầu vào, ResNet18 Encoder tạo ra feature map có dạng:

$$F \in \mathbb{R}^{B \times C \times H' \times W'}$$

Trong đó:

- B là batch size.
- C là số kênh đặc trưng.
- H' là chiều cao của feature map.
- W' là chiều rộng của feature map.

Do chữ viết trong ảnh thường được đọc từ trái sang phải, chiều rộng W' của feature map có thể được xem như trục thời gian. Feature map sau đó được reshape thành chuỗi đặc trưng để đưa vào BiLSTM hoặc decoder.

2.4. Mạng hồi quy và BiLSTM

Recurrent Neural Network (RNN) là nhóm mô hình được thiết kế để xử lý dữ liệu tuần tự. Trong bài toán nhận dạng chữ viết tay, sau khi ảnh được trích xuất đặc trưng bởi CNN, các đặc trưng dọc theo chiều ngang ảnh có thể được xem như một chuỗi. Mỗi phần tử trong chuỗi tương ứng với một vùng đặc trưng của ảnh.

Tuy nhiên, RNN truyền thống gặp khó khăn khi học các phụ thuộc dài hạn do hiện tượng vanishing gradient. Long Short-Term Memory (LSTM) được đề xuất để cải thiện khả năng ghi nhớ thông tin dài hạn thông qua các cổng điều khiển. LSTM có khả năng quyết định thông tin nào cần giữ lại, thông tin nào cần quên và thông tin nào cần đưa ra đầu ra tại mỗi bước thời gian.

BiLSTM là biến thể hai chiều của LSTM. Thay vì chỉ xử lý chuỗi theo chiều trái sang phải, BiLSTM xử lý chuỗi theo cả hai chiều:

- Chiều thuận: từ trái sang phải.
- Chiều ngược: từ phải sang trái.

Đầu ra của BiLSTM tại mỗi vị trí là sự kết hợp thông tin từ cả hai hướng. Điều này đặc biệt hữu ích trong nhận dạng chữ viết tay vì một ký tự có thể được nhận diện tốt hơn khi xét cả ngữ cảnh phía trước và phía sau.

Trong dự án, BiLSTM được sử dụng ở cả hai mô hình:

- Trong CTC Baseline, BiLSTM xử lý chuỗi đặc trưng trước khi dự đoán phân phối ký tự tại từng bước thời gian.
- Trong Attention Model, BiLSTM đóng vai trò Context Encoder, giúp đặc trưng đưa vào attention giàu thông tin ngữ cảnh hơn.

2.5. Connectionist Temporal Classification

Connectionist Temporal Classification (CTC) là một hàm mất mát thường được sử dụng cho các bài toán nhận dạng chuỗi khi không có thông tin căn chỉnh giữa đầu vào và đầu ra. Trong nhận dạng chữ viết tay, ta biết một ảnh tương ứng với một từ, nhưng không biết chính xác mỗi ký tự nằm ở vị trí nào trong ảnh.

CTC giải quyết vấn đề này bằng cách cho phép mô hình sinh ra một chuỗi trung gian dài hơn chuỗi nhãn thật. Chuỗi trung gian này có thể chứa các ký tự lặp và một ký hiệu đặc biệt gọi là blank token. Sau đó, chuỗi trung gian được rút gọn thông qua hai bước:

1. Gộp các ký tự giống nhau đứng liên tiếp.
2. Loại bỏ blank token.

Ví dụ, giả sử ký hiệu – là blank token, chuỗi dự đoán thô:

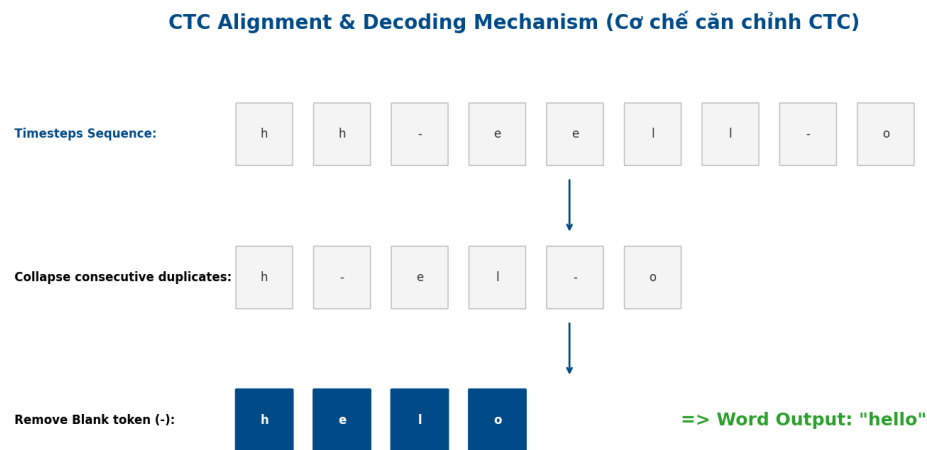
–hh–e–ll–oo–

có thể được rút gọn thành:

helo

CTC Loss tính tổng xác suất của tất cả các chuỗi căn chỉnh hợp lệ có thể rút gọn về nhãn đúng. Nhờ đó, mô hình có thể được huấn luyện mà không cần gán nhãn vị trí từng ký tự trên ảnh.

Ưu điểm của CTC là đơn giản, ổn định và phù hợp với các bài toán có thứ tự đầu ra gần như đơn điệu theo đầu vào. Tuy nhiên, CTC cũng có một số hạn chế. Mô hình thường giả định các dự đoán tại từng bước thời gian độc lập có điều kiện với nhau, nên khó khai thác trực tiếp quan hệ giữa các ký tự đã sinh trước đó. Ngoài ra, độ dài chuỗi đầu ra thường bị giới hạn bởi số timestep của feature map.



Hình 2.2: Minh họa ý tưởng căn chỉnh trong CTC

2.6. Cơ chế Attention

Attention là cơ chế cho phép mô hình tập trung vào các phần khác nhau của đầu vào khi sinh từng phần tử đầu ra. Thay vì nén toàn bộ đầu vào vào một vector cố định, attention cho phép decoder truy cập lại các đặc trưng encoder ở từng bước giải mã.

Trong bài toán nhận dạng chữ viết tay, khi mô hình dự đoán một ký tự, attention có thể học cách tập trung vào vùng ảnh tương ứng với ký tự đó. Điều này tạo ra một dạng căn chỉnh mềm giữa chuỗi đặc trưng ảnh và chuỗi ký tự đầu ra.

Dự án sử dụng Bahdanau Attention, còn gọi là additive attention. Tại bước giải mã t , attention tính điểm liên quan giữa trạng thái decoder trước đó s_{t-1} và từng vector đặc trưng encoder h_s :

$$e_{t,s} = v_a^T \tanh(W_a s_{t-1} + U_a h_s)$$

Sau đó, các điểm này được chuẩn hóa bằng softmax để tạo ra trọng số attention:

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{k=1}^T \exp(e_{t,k})}$$

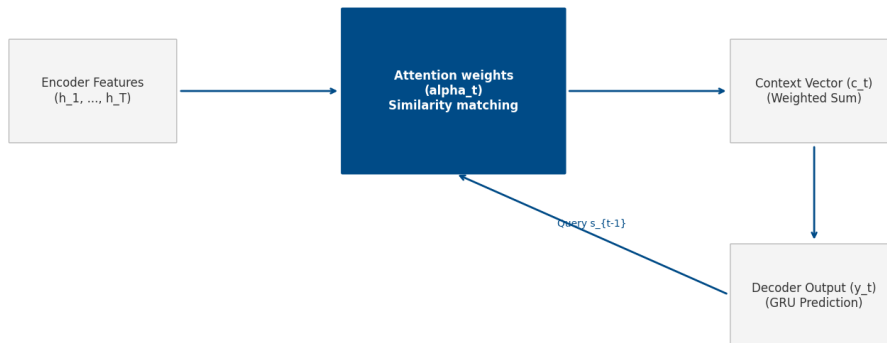
Vector ngữ cảnh tại bước t được tính bằng tổng có trọng số của các vector encoder:

$$c_t = \sum_{s=1}^T \alpha_{t,s} h_s$$

Trong đó:

- h_s là đặc trưng encoder tại vị trí s .
- s_{t-1} là trạng thái ẩn của decoder ở bước trước.
- $\alpha_{t,s}$ là mức độ chú ý của decoder tại bước t đối với vị trí encoder s .
- c_t là vector ngữ cảnh đưa vào decoder để dự đoán ký tự tiếp theo.

Bahdanau Attention Mechanism (Cơ chế chú ý Bahdanau)



Hình 2.3: Minh họa cơ chế attention trong mô hình Encoder–Decoder

2.7. Kiến trúc Encoder–Decoder

Encoder–Decoder là kiến trúc thường được sử dụng trong các bài toán chuyển đổi chuỗi, ví dụ như dịch máy, tóm tắt văn bản hoặc nhận dạng văn bản từ ảnh. Encoder có nhiệm vụ mã hóa đầu vào thành một biểu diễn đặc trưng, còn decoder sinh đầu ra từng bước dựa trên biểu diễn đó.

Trong dự án này, encoder gồm CNN Encoder và BiLSTM Context Encoder. CNN Encoder trích xuất đặc trưng không gian từ ảnh. Sau đó, feature map được reshape thành chuỗi và đưa qua BiLSTM để bổ sung thông tin ngữ cảnh hai chiều.

Decoder sử dụng GRUCell để sinh chuỗi ký tự đầu ra. Tại mỗi bước giải mã, decoder nhận embedding của ký tự trước đó, vector ngữ cảnh từ attention và trạng thái ẩn hiện tại để dự đoán ký tự tiếp theo.

Mô hình Encoder–Decoder với Attention có ưu điểm là linh hoạt hơn CTC. Decoder có thể điều kiện hóa ký tự hiện tại dựa trên các ký tự đã sinh trước đó. Điều này giúp mô hình học được một phần quan hệ ngôn ngữ giữa các ký tự trong từ. Ngoài ra, attention map cũng có thể được trực quan hóa để quan sát vùng ảnh mà mô hình tập trung trong quá trình dự đoán.

2.8. Teacher Forcing

Teacher Forcing là một kỹ thuật thường được sử dụng khi huấn luyện các mô hình decoder tự hồi quy. Trong quá trình sinh chuỗi, decoder cần sử dụng ký tự ở bước trước làm đầu vào cho bước hiện tại. Có hai lựa chọn:

- Sử dụng ký tự mà mô hình đã dự đoán ở bước trước.
- Sử dụng ký tự đúng từ nhãn ground truth.

Teacher Forcing sử dụng ký tự đúng từ nhãn ground truth trong một số bước huấn luyện. Điều này giúp mô hình học nhanh hơn và giảm khả năng lỗi ở bước trước lan truyền sang các bước sau trong giai đoạn đầu huấn luyện.

Trong dự án, Teacher Forcing được sử dụng cho Attention Model trong quá trình train. Khi đánh giá và suy diễn, Teacher Forcing không được sử dụng vì mô hình phải tự sinh toàn bộ chuỗi dự đoán.

2.9. Các phương pháp giải mã

Sau khi mô hình tạo ra phân phối xác suất trên các ký tự, cần có thuật toán giải mã để chuyển các phân phối này thành chuỗi văn bản cuối cùng.

2.9.1. CTC Greedy Decoding

Đối với mô hình CTC, phương pháp giải mã đơn giản nhất là CTC Greedy Decoding. Tại mỗi timestep, mô hình chọn ký tự có xác suất cao nhất. Sau đó, chuỗi thô được rút gọn bằng cách loại bỏ ký tự lặp liên tiếp và blank token.

Phương pháp này có tốc độ nhanh và dễ triển khai, nhưng có thể bỏ qua những chuỗi có xác suất tổng thể tốt hơn.

2.9.2. Attention Greedy Decoding

Với mô hình Attention, Greedy Decoding sinh chuỗi từng ký tự. Tại mỗi bước, decoder chọn ký tự có xác suất cao nhất làm đầu ra. Ký tự này sau đó được đưa vào bước giải mã tiếp theo.

Greedy Decoding đơn giản và phù hợp cho suy diễn nhanh, nhưng có thể bị ảnh hưởng nếu mô hình chọn sai ký tự ở một bước nào đó.

2.9.3. Beam Search

Beam Search mở rộng Greedy Decoding bằng cách giữ lại nhiều chuỗi ứng viên tốt nhất tại mỗi bước thay vì chỉ giữ một chuỗi duy nhất. Số lượng ứng viên được giữ lại gọi là beam size.

Beam Search có thể tìm được chuỗi dự đoán tốt hơn Greedy Decoding, nhưng chi phí tính toán cao hơn. Trong dự án, Beam Search được triển khai cho mô hình Attention để hỗ trợ suy diễn nâng cao.

2.10. Các chỉ số đánh giá

Để đánh giá mô hình nhận dạng chữ viết tay, dự án sử dụng ba chỉ số chính: Word Accuracy, Character Error Rate và Normalized Edit Distance.

2.10.1. Word Accuracy

Word Accuracy đo tỷ lệ số từ được dự đoán chính xác hoàn toàn:

$$WA = \frac{\text{Số từ dự đoán đúng hoàn toàn}}{\text{Tổng số từ kiểm thử}} \times 100\%$$

Đây là chỉ số nghiêm ngặt. Chỉ cần một ký tự trong từ bị sai, toàn bộ từ đó được xem là dự đoán sai.

2.10.2. Character Error Rate

Character Error Rate (CER) đo tỷ lệ lỗi ở mức ký tự dựa trên edit distance:

$$CER = \frac{\sum \text{edit_distance}(\text{pred}, \text{target})}{\sum \text{len}(\text{target})} \times 100\%$$

CER cho biết mức độ sai khác giữa chuỗi dự đoán và chuỗi nhãn thật ở cấp độ ký tự. Chỉ số này hữu ích vì trong nhiều trường hợp, mô hình có thể dự đoán sai cả từ nhưng chỉ sai một hoặc hai ký tự.

2.10.3. Normalized Edit Distance

Normalized Edit Distance (NED) là chỉ số dựa trên edit distance đã được chuẩn hóa theo độ dài chuỗi:

$$NED = 1 - \frac{1}{N} \sum_{i=1}^N \frac{\text{edit_distance}(\text{pred}_i, \text{target}_i)}{\max(\text{len}(\text{pred}_i), \text{len}(\text{target}_i))}$$

NED càng cao thì chuỗi dự đoán càng gần với nhãn đúng. Chỉ số này giúp đánh giá mức độ tương đồng giữa dự đoán và ground truth theo từng mẫu.

2.11. Tổng kết chương

Chương này đã trình bày các cơ sở lý thuyết chính liên quan đến dự án, bao gồm bài toán nhận dạng chữ viết tay, CNN, ResNet18, BiLSTM, CTC, Attention, Encoder–Decoder, Teacher Forcing, các phương pháp giải mã và các chỉ số đánh giá. Đây là nền tảng để xây dựng và phân tích hai mô hình được triển khai trong các chương tiếp theo.

CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1. Giới thiệu bộ dữ liệu IAM

Dự án sử dụng IAM Handwriting Dataset ở mức word-level. Đây là bộ dữ liệu chữ viết tay tiếng Anh được sử dụng phổ biến trong các nghiên cứu về nhận dạng chữ viết tay ngoại tuyến. Dữ liệu word-level gồm các ảnh đã được tách sẵn, mỗi ảnh chứa một từ viết tay và có nhãn văn bản tương ứng.

Trong phạm vi dự án, nhóm sử dụng dữ liệu ở mức từ đơn lẻ thay vì mức dòng hoặc mức văn bản hoàn chỉnh. Cách tiếp cận này phù hợp với mục tiêu của bài tập lớn vì giúp nhóm tập trung vào bài toán nhận dạng chuỗi ký tự từ ảnh mà không cần xử lý thêm các bước phức tạp như phát hiện dòng, tách từ hoặc phân tích bố cục tài liệu.

Sau quá trình lọc và làm sạch nhãn, tổng số mẫu được sử dụng trong dự án là 96,835 mẫu. Mỗi mẫu gồm một ảnh chữ viết tay và một nhãn là từ tiếng Anh tương ứng.



Hình 3.1: Một số ảnh mẫu từ IAM Handwriting Dataset ở mức word-level

3.2. Đặc điểm dữ liệu

Dữ liệu chữ viết tay trong IAM có nhiều đặc điểm khiến bài toán nhận dạng trở nên khó hơn so với nhận dạng chữ in. Một số đặc điểm chính gồm:

- Nét chữ giữa các người viết khác nhau có độ biến thiên lớn.
- Một số ký tự có hình dạng gần giống nhau, ví dụ o và a, r và v, hoặc l và i.
- Độ dài các từ không cố định.
- Một số ảnh có độ nghiêng, độ mờ hoặc khoảng cách ký tự không đồng đều.

- Ảnh có thể chứa vùng trắng padding hoặc khoảng trống lớn hai bên chữ.

Những đặc điểm này đòi hỏi hệ thống cần có khả năng trích xuất đặc trưng ảnh tốt và mô hình hóa được quan hệ tuần tự giữa các ký tự trong từ.

3.3. Cấu trúc dữ liệu đầu vào

Dữ liệu đầu vào của dự án được tổ chức thành thư mục ảnh và file nhãn. Thư mục `words/` chứa các ảnh chữ viết tay, trong khi file `label.txt` lưu thông tin nhãn tương ứng với từng ảnh.

Cấu trúc dữ liệu tổng quát có dạng:

```
1 dataset/  
2     words/  
3         image_001.png  
4         image_002.png  
5         image_003.png  
6         ...  
7     label.txt
```

Mỗi dòng trong file nhãn ánh xạ một ảnh với nội dung văn bản tương ứng. Trong quá trình chuẩn bị dữ liệu, hệ thống đọc file nhãn, kiểm tra sự tồn tại của ảnh, xử lý nhãn và loại bỏ các mẫu không hợp lệ.

Sau bước chuẩn bị dữ liệu, hệ thống sinh ra các file CSV tương ứng với các tập train, validation và test. Các file này được sử dụng trực tiếp trong quá trình huấn luyện và đánh giá mô hình.

3.4. Làm sạch nhãn

Dữ liệu nhãn ban đầu có thể chứa nhiều loại ký tự khác nhau. Để đơn giản hóa bài toán và thống nhất không gian đầu ra, nhóm thực hiện làm sạch nhãn trước khi huấn luyện mô hình.

Các bước làm sạch nhãn gồm:

- Chuyển toàn bộ nhãn về chữ thường.
- Loại bỏ các ký tự không thuộc phạm vi từ vựng được hỗ trợ.
- Chỉ giữ lại các mẫu có nhãn hợp lệ sau khi làm sạch.
- Loại bỏ các mẫu ảnh lỗi hoặc không đọc được.
- Chuẩn hóa nhãn để phục vụ quá trình mã hóa ký tự.

Trong dự án, nhóm tập trung vào các từ tiếng Anh gồm các ký tự chữ cái thường. Việc giới hạn không gian ký tự giúp quá trình huấn luyện ổn định hơn và giảm độ phức tạp của bài toán.

3.5. Xây dựng bộ từ vựng ký tự

Sau khi làm sạch nhãn, hệ thống xây dựng bộ từ vựng ký tự dùng cho quá trình encode và decode. Mỗi ký tự được ánh xạ thành một chỉ số nguyên. Tùy theo mô hình, bộ từ vựng có thể bổ sung thêm các token đặc biệt.

Đối với mô hình CTC, cần có blank token để phục vụ CTC Loss. Blank token thường được gán chỉ số 0. Các ký tự còn lại được ánh xạ vào các chỉ số tiếp theo.

Đối với mô hình Attention, cần có các token đặc biệt như:

- `<pad>`: dùng để padding các nhãn trong cùng batch.
- `<sos>`: ký hiệu bắt đầu chuỗi.
- `<eos>`: ký hiệu kết thúc chuỗi.

Việc mã hóa nhãn giúp chuyển chuỗi văn bản thành tensor số để mô hình có thể xử lý trong quá trình huấn luyện.

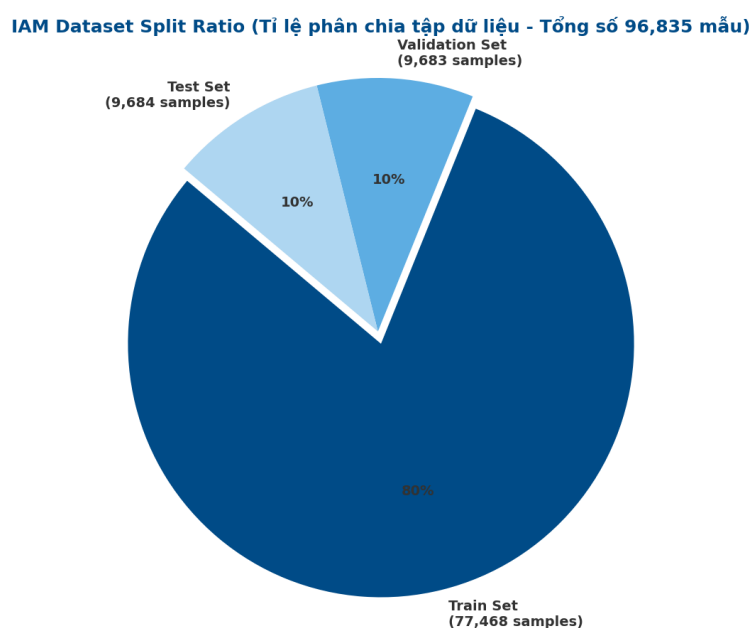
3.6. Chia tập dữ liệu

Sau khi lọc và làm sạch, dữ liệu được chia thành ba tập: train, validation và test. Tỷ lệ chia là 80/10/10. Kết quả phân chia được trình bày trong Bảng 3.1.

Bảng 3.1: Phân chia bộ dữ liệu IAM sau tiền xử lý

Tập dữ liệu	Số mẫu	Tỷ lệ
Train Set	77,468	80%
Validation Set	9,683	10%
Test Set	9,684	10%
Tổng cộng	96,835	100%

Tập train được sử dụng để cập nhật trọng số mô hình. Tập validation được sử dụng để theo dõi quá trình huấn luyện và lựa chọn checkpoint tốt nhất. Tập test chỉ được sử dụng ở bước đánh giá cuối cùng nhằm phản ánh khách quan hơn hiệu quả của mô hình sau khi huấn luyện.



Hình 3.2: Tỷ lệ phân chia dữ liệu train, validation và test

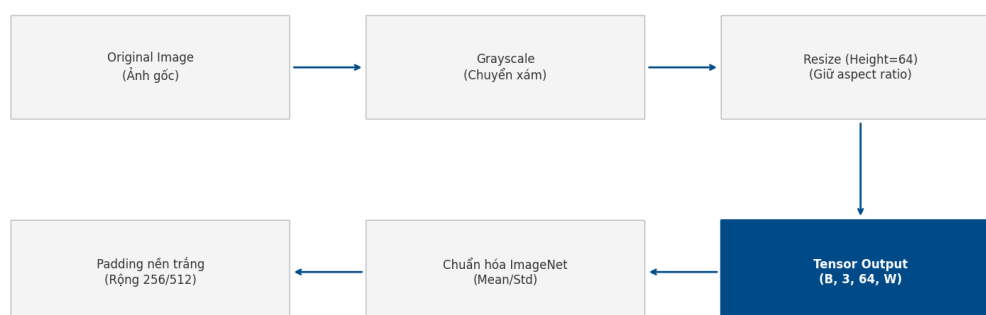
3.7. Tiền xử lý ảnh

Tiền xử lý ảnh là bước quan trọng vì chất lượng ảnh đầu vào ảnh hưởng trực tiếp đến khả năng học của mô hình. Trong dự án, ảnh được xử lý theo một pipeline thống nhất trước khi đưa vào mô hình.

Các bước tiền xử lý chính gồm:

1. Đọc ảnh từ đường dẫn tương ứng.
2. Chuyển ảnh về grayscale nếu cần thiết.
3. Resize ảnh về chiều cao cố định.
4. Giữ tỉ lệ khía cạnh của ảnh khi resize.
5. Padding nền trắng về bên phải nếu ảnh có chiều rộng nhỏ hơn chiều rộng mục tiêu.
6. Crop hoặc giới hạn chiều rộng nếu ảnh vượt quá kích thước mục tiêu.
7. Nhân bản ảnh grayscale thành 3 kênh để tương thích với ResNet18.
8. Chuẩn hóa giá trị pixel và chuyển ảnh thành tensor.

Preprocessing Pipeline (Các bước Tiền xử lý ảnh)



Hình 3.3: Pipeline tiền xử lý ảnh đầu vào

3.8. Kích thước ảnh đầu vào

Trong thiết kế hệ thống và cấu hình thực nghiệm, hai mô hình có thể được thiết lập với kích thước ảnh đầu vào khác nhau. Theo cấu hình thiết kế tiêu chuẩn:

- Với mô hình CTC Baseline, ảnh được resize và padding về kích thước 64×256 , sau khi đi qua ResNet18 Encoder, feature map được reshape thành chuỗi có độ dài thời gian $T = 16$.
- Với mô hình Attention, ảnh được resize và padding về kích thước 64×512 , sau khi đi qua ResNet18 Encoder, feature map được reshape thành chuỗi có độ dài thời gian $T = 32$.

Trong mã nguồn cài đặt thực tế, hệ thống hỗ trợ cấu hình động thông qua file `config.py` (mặc định đang thiết lập chung ở chiều rộng 512 để tối ưu hóa khả năng nhận dạng của mô hình Attention, và CTC Baseline có thể tự động thích ứng với kích thước này nhờ tính độc lập về chiều rộng của mạng tích chập và LSTM).

Bảng 3.2 trình bày sự khác biệt về kích thước đầu vào giữa hai mô hình.

Bảng 3.2: Kích thước ảnh đầu vào của hai mô hình

Mô hình	Kích thước ảnh	Feature map sau CNN	Độ dài chuỗi
CTC Baseline	64×256	$(B, 256, 4, 16)$	$T = 16$
Attention Model	64×512	$(B, 256, 4, 32)$	$T = 32$

Việc sử dụng chiều rộng lớn hơn cho Attention Model giúp mô hình có nhiều vị trí đặc trưng hơn để attention tập trung trong quá trình sinh ký tự. Điều này phù hợp với đặc điểm của kiến trúc Encoder–Decoder có cơ chế attention.

3.9. Chuẩn hóa ảnh

Do ResNet18 thường được huấn luyện trước trên ImageNet, ảnh đầu vào được chuẩn hóa theo trung bình và độ lệch chuẩn của ImageNet:

$$\mu = [0.485, 0.456, 0.406]$$

$$\sigma = [0.229, 0.224, 0.225]$$

Mặc dù ảnh chữ viết tay ban đầu là grayscale, ảnh được nhân bản thành 3 kênh trước khi đưa vào ResNet18. Điều này giúp tận dụng được encoder có sẵn mà không cần thay đổi hoàn toàn kiến trúc đầu vào.

3.10. Tăng cường dữ liệu

Tăng cường dữ liệu được sử dụng trong quá trình huấn luyện nhằm giúp mô hình tiếp xúc với nhiều biến thể ảnh hơn. Điều này giúp giảm overfitting và cải thiện khả năng tổng quát hóa.

Các kỹ thuật tăng cường dữ liệu được sử dụng gồm:

- **Random Rotation:** xoay ảnh ngẫu nhiên trong khoảng $[-15^\circ, 15^\circ]$.
- **Affine/Shear Transform:** mô phỏng độ nghiêng của chữ viết.
- **Elastic Transform:** mô phỏng sự biến dạng tự nhiên của nét chữ.
- **Gaussian Blur:** mô phỏng ảnh bị mờ.
- **Gaussian Noise:** mô phỏng nhiễu trong ảnh.
- **Random Brightness/Contrast:** thay đổi độ sáng và tương phản để mô phỏng điều kiện ảnh khác nhau.

Data Augmentation Techniques (Các phép tăng cường dữ liệu)



Hình 3.4: Ví dụ minh họa một số phép tăng cường dữ liệu

Các phép tăng cường dữ liệu được áp dụng chủ yếu cho tập train. Tập validation và test không áp dụng augmentation để đảm bảo quá trình đánh giá phản ánh đúng khả năng mô hình trên dữ liệu gốc.

3.11. Xây dựng Dataset và DataLoader

Dự án sử dụng Dataset và DataLoader của PyTorch để quản lý quá trình đọc dữ liệu và tạo batch. Dataset chịu trách nhiệm đọc ảnh, áp dụng tiền xử lý, mã hóa nhãn và trả về tensor tương ứng. DataLoader chịu trách nhiệm gom mẫu thành batch, shuffle dữ liệu huấn luyện và hỗ trợ quá trình train hiệu quả hơn.

Vì các nhãn có độ dài khác nhau, hệ thống cần xử lý padding khi tạo batch. Đối với Attention Model, các chuỗi nhãn được padding bằng token <pad> để có cùng độ dài trong batch. Trong quá trình tính loss, các vị trí padding được bỏ qua bằng `ignore_index`.

Đối với CTC Baseline, ngoài tensor ảnh và nhãn đã encode, mô hình cần thêm độ dài thực tế của chuỗi nhãn và độ dài chuỗi đầu vào. Các thông tin này được sử dụng bởi CTC Loss để tính xác suất trên tất cả các alignment hợp lệ.

3.12. Pipeline chuẩn bị dữ liệu

Toàn bộ quá trình chuẩn bị dữ liệu được thực hiện thông qua script:

```
python -m src.prepare_data
```

Script này thực hiện các bước:

1. Đọc dữ liệu gốc từ thư mục `dataset/`.
2. Đọc file `label.txt`.
3. Kiểm tra ảnh tồn tại và có thể đọc được.
4. Làm sạch nhãn.

5. Lọc các mẫu không hợp lệ.
6. Chia dữ liệu thành train, validation và test.
7. Lưu các file CSV đã xử lý để sử dụng trong huấn luyện.

Quy trình này giúp tách riêng bước chuẩn bị dữ liệu khỏi bước huấn luyện mô hình, giúp hệ thống dễ kiểm soát và dễ tái lập hơn.

3.13. Một số hạn chế của dữ liệu

Mặc dù IAM là bộ dữ liệu chuẩn và phù hợp cho bài toán nhận dạng chữ viết tay, dữ liệu này vẫn có một số hạn chế khi so sánh với ảnh thực tế do người dùng tự chụp hoặc tự viết.

Một số hạn chế gồm:

- Ảnh trong IAM thường có chất lượng và cách cắt tương đối chuẩn hơn ảnh thực tế.
- Dữ liệu chủ yếu là chữ viết tay tiếng Anh, chưa bao phủ nhiều ngữ cảnh ngôn ngữ khác.
- Ảnh thực tế có thể có nền phức tạp, ánh sáng không đều hoặc góc chụp lệch.
- Nét chữ cá nhân ngoài IAM có thể khác đáng kể so với phân phối huấn luyện.

Do đó, kết quả tốt trên tập test IAM không đảm bảo mô hình sẽ hoạt động ổn định với mọi ảnh chữ viết tay thực tế. Đây là một trong những nguyên nhân chính khiến hệ thống cần tiếp tục được cải thiện nếu triển khai trong môi trường thực tế.

3.14. Tổng kết chương

Chương này đã trình bày bộ dữ liệu được sử dụng trong dự án, cấu trúc dữ liệu đầu vào, quá trình làm sạch nhân, xây dựng vocabulary, chia tập dữ liệu, tiền xử lý ảnh, tăng cường dữ liệu và xây dựng DataLoader. Đây là các bước nền tảng để đảm bảo dữ liệu đầu vào phù hợp với quá trình huấn luyện và đánh giá hai mô hình CTC Baseline và Attention Model.

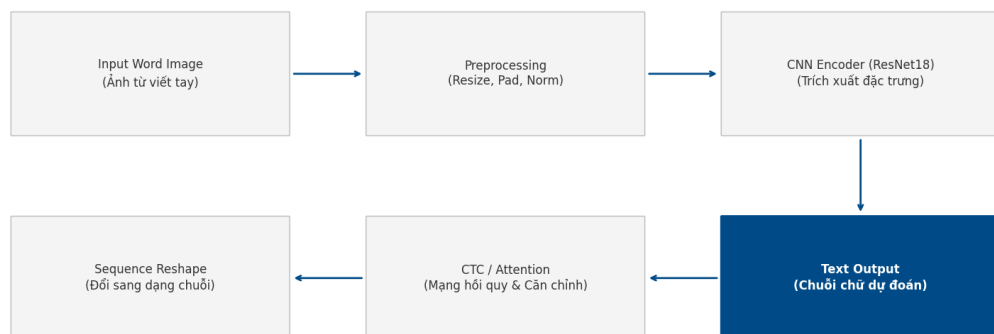
CHƯƠNG 4. KIẾN TRÚC MÔ HÌNH

4.1. Tổng quan kiến trúc hệ thống

Dự án xây dựng hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ theo hướng học sâu. Đầu vào của hệ thống là một ảnh chứa duy nhất một từ tiếng Anh viết tay. Đầu ra là chuỗi ký tự dự đoán tương ứng với nội dung trong ảnh.

Pipeline tổng quát của hệ thống gồm các bước chính: tiền xử lý ảnh, trích xuất đặc trưng bằng CNN, chuyển đổi feature map thành chuỗi đặc trưng, xử lý chuỗi bằng mô hình tuần tự, giải mã và sinh chuỗi ký tự đầu ra.

Overall HTR System Pipeline (Sơ đồ luồng xử lý hệ thống)



Hình 4.1: Pipeline tổng quan của hệ thống nhận dạng chữ viết tay

Về mặt kiến trúc mô hình, dự án triển khai và so sánh hai hướng tiếp cận chính:

- **CTC Baseline:** sử dụng ResNet18 làm CNN Encoder, BiLSTM để xử lý chuỗi đặc trưng và CTC Loss để học cân chỉnh giữa ảnh đầu vào và chuỗi ký tự đầu ra.
- **Attention Model:** sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder để sinh chuỗi ký tự theo cơ chế Encoder–Decoder.

Hai mô hình đều sử dụng ResNet18 để trích xuất đặc trưng ảnh, nhưng khác nhau ở cách xử lý chuỗi và cách sinh đầu ra. Mô hình CTC dự đoán phân phối ký tự tại từng bước thời gian và dùng CTC Decoding để tạo chuỗi cuối cùng. Mô hình Attention sinh ký tự từng bước thông qua decoder tự hồi quy.

4.2. CNN Encoder sử dụng ResNet18

4.2.1. Vai trò của CNN Encoder

Trong bài toán nhận dạng chữ viết tay, ảnh đầu vào chứa thông tin không gian của các nét chữ. Để mô hình có thể xử lý hiệu quả, ảnh cần được chuyển đổi thành biểu diễn đặc trưng có ý nghĩa hơn so với pixel thô. CNN Encoder đảm nhiệm vai trò trích xuất đặc trưng này.

Dự án sử dụng ResNet18 làm CNN Encoder. ResNet18 có ưu điểm là kiến trúc tương đối gọn, dễ triển khai và có khả năng trích xuất đặc trưng ảnh tốt. Thay vì sử dụng ResNet18 như một mô hình phân loại ảnh hoàn chỉnh, dự án chỉ sử dụng các tầng tích chập của mạng để sinh feature map.

4.2.2. Đầu vào và đầu ra của Encoder

Ảnh đầu vào sau tiền xử lý được đưa về kích thước chuẩn. Trong cấu hình thực nghiệm và thiết kế lý thuyết của nhóm:

- CTC Baseline sử dụng ảnh kích thước 64×256 làm thiết lập chuẩn.
- Attention Model sử dụng ảnh kích thước 64×512 làm thiết lập chuẩn.

Sau khi đi qua ResNet18 Encoder, ảnh được biến đổi thành feature map có dạng:

$$F \in \mathbb{R}^{B \times C \times H' \times W'}$$

Trong đó B là batch size, C là số kênh đặc trưng, H' là chiều cao feature map và W' là chiều rộng feature map.

Với cấu hình thiết kế lý thuyết, đầu ra của Encoder có dạng:

- CTC Baseline (ảnh 64×256): feature map xấp xỉ $(B, 256, 4, 16)$, tương ứng với độ dài chuỗi đặc trưng $T = 16$.
- Attention Model (ảnh 64×512): feature map xấp xỉ $(B, 256, 4, 32)$, tương ứng với độ dài chuỗi đặc trưng $T = 32$.

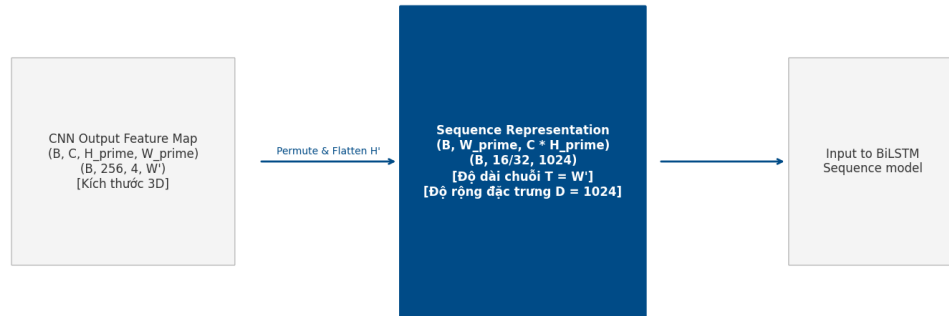
Trong thực tế cài đặt mã nguồn, do mạng CNN hoạt động độc lập với chiều rộng ảnh đầu vào, cả hai mô hình đều có thể được chạy chung trên cấu hình chiều rộng mặc định là 512 (độ dài chuỗi $T = 32$) thông qua file `config.py` để đơn giản hóa pipeline dữ liệu.

4.3. Chuyển đổi Feature Map thành chuỗi đặc trưng

Sau khi đi qua CNN Encoder, feature map vẫn là biểu diễn hai chiều. Tuy nhiên, nhận dạng chữ viết tay là bài toán sinh chuỗi ký tự. Do đó, feature map cần được chuyển đổi thành chuỗi đặc trưng theo chiều ngang của ảnh.

Vì chữ tiếng Anh thường được viết và đọc từ trái sang phải, chiều rộng của feature map có thể được xem như trục thời gian. Tại mỗi vị trí theo chiều ngang, mô hình sử dụng đặc trưng tương ứng với một vùng của ảnh.

Feature Reshape Process (Quy trình biến đổi đặc trưng không gian thành chuỗi)



Hình 4.2: Minh họa quá trình chuyển feature map 2D thành chuỗi đặc trưng

Với CTC Baseline, quá trình reshape có dạng:

$$(B, 256, 4, 16) \rightarrow (B, 16, 1024)$$

Trong đó chuỗi đầu ra có độ dài $T = 16$, mỗi bước thời gian có vector đặc trưng 1024 chiều.

Với Attention Model, quá trình reshape có dạng:

$$(B, 256, 4, 32) \rightarrow (B, 32, 1024)$$

Trong đó chuỗi đầu ra có độ dài $T = 32$. Chuỗi dài hơn giúp cơ chế attention có nhiều vị trí để tập trung hơn khi sinh từng ký tự.

4.4. Mô hình CTC Baseline

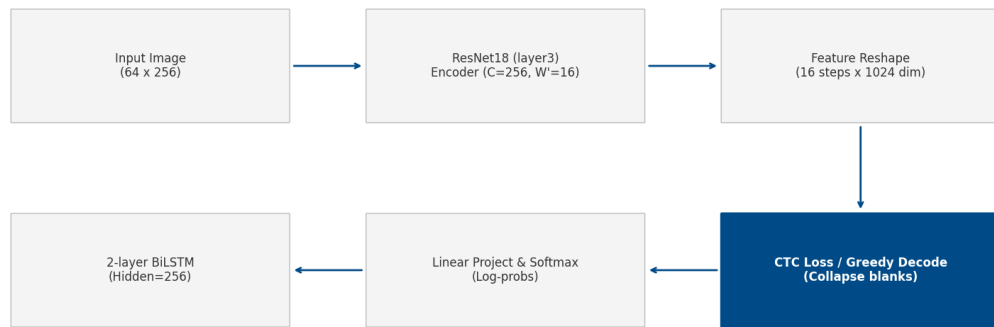
4.4.1. Tổng quan mô hình

CTC Baseline là mô hình cơ sở được xây dựng nhằm tạo mức chuẩn để so sánh với mô hình Attention. Kiến trúc này có ưu điểm là đơn giản, trực tiếp và phù hợp với bài toán nhận dạng chuỗi khi không có thông tin căn chỉnh cụ thể giữa ảnh và ký tự.

Pipeline của mô hình CTC Baseline gồm:

Image $\rightarrow$ ResNet18 Encoder $\rightarrow$ Sequence Reshape $\rightarrow$ BiLSTM $\rightarrow$ Linear Projection $\rightarrow$ CTC Loss

CTC Baseline Architecture (Kiến trúc mô hình CTC)



Hình 4.3: Kiến trúc mô hình CTC Baseline

4.4.2. BiLSTM trong CTC Baseline

Sau khi feature map được chuyển thành chuỗi đặc trưng, chuỗi này được đưa qua BiLSTM. BiLSTM xử lý chuỗi theo cả hai chiều trái sang phải và phải sang trái. Nhờ đó, mỗi vị trí trong chuỗi đầu ra không chỉ chứa thông tin tại vùng ảnh hiện tại mà còn chứa ngữ cảnh từ các vùng ảnh xung quanh.

Trong mô hình CTC Baseline, BiLSTM gồm 2 lớp với hidden dimension bằng 256 cho mỗi chiều. Vì có hai chiều, vector đầu ra tại mỗi timestep có kích thước 512. Dropout được sử dụng để giảm overfitting trong quá trình huấn luyện.

4.4.3. Linear Projection

Đầu ra của BiLSTM được đưa qua một lớp tuyến tính để ánh xạ từ không gian đặc trưng sang không gian ký tự. Nếu vocabulary có V ký tự, đầu ra tại mỗi timestep là một vector có kích thước V .

Sau đó, mô hình áp dụng `log_softmax` để tạo phân phối xác suất logarit trên các ký tự. Kết quả này được đưa vào CTC Loss trong quá trình huấn luyện.

4.4.4. CTC Loss

CTC Loss cho phép huấn luyện mô hình mà không cần biết vị trí chính xác của từng ký tự trong ảnh. Với mỗi ảnh đầu vào, mô hình sinh ra một chuỗi phân phối xác suất có độ dài cố định theo số timestep. Trong khi đó, nhãn thật là chuỗi ký tự có độ dài thay đổi.

CTC xem xét tất cả các căn chỉnh hợp lệ có thể rút gọn về nhãn đúng, sau đó tối đa hóa tổng xác suất của các căn chỉnh đó. Nhờ vậy, mô hình có thể học ánh xạ từ chuỗi đặc trưng ảnh sang chuỗi ký tự mà không cần dữ liệu gán nhãn ở mức vị trí ký tự.

4.4.5. CTC Greedy Decoding

Khi suy diễn, mô hình CTC Baseline sử dụng CTC Greedy Decoding. Quá trình này gồm ba bước:

1. Tại mỗi timestep, chọn ký tự có xác suất cao nhất.
2. Gộp các ký tự giống nhau đứng liên tiếp.
3. Loại bỏ blank token để thu được chuỗi cuối cùng.

Ví dụ, với blank token được biểu diễn là -, chuỗi thô:

-hhe-ll-oo-

sau khi xử lý có thể trở thành:

helo

Phương pháp này có ưu điểm là nhanh và dễ triển khai, phù hợp để đánh giá baseline. Tuy nhiên, greedy decoding có thể bỏ qua những chuỗi có xác suất tổng thể tốt hơn nếu tại một bước nào đó lựa chọn cục bộ không tối ưu.

4.5. Mô hình Attention

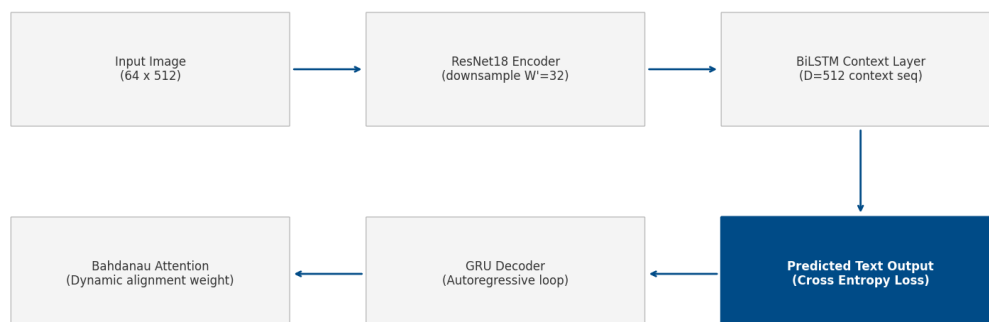
4.5.1. Tổng quan mô hình

Mô hình Attention là kiến trúc chính của dự án, được xây dựng theo hướng Encoder-Decoder. Khác với CTC Baseline, mô hình Attention sử dụng decoder tự hồi quy để sinh chuỗi ký tự từng bước.

Pipeline của mô hình Attention gồm:

Image → ResNet18 Encoder → Sequence Reshape → BiLSTM Context Encoder → Bahdanau Attention → GRU Decoder

Attention Model Architecture (Kiến trúc mô hình Attention)



Hình 4.4: Kiến trúc mô hình Attention

4.5.2. BiLSTM Context Encoder

Sau khi ResNet18 trích xuất feature map và feature map được reshape thành chuỗi, mô hình đưa chuỗi này qua BiLSTM Context Encoder. Khối BiLSTM này có nhiệm vụ bổ sung thông tin ngữ cảnh hai chiều cho từng vị trí đặc trưng.

Nếu chỉ sử dụng đặc trưng CNN cục bộ, mỗi vị trí trong chuỗi đặc trưng có thể thiếu thông tin liên quan đến các ký tự xung quanh. Điều này làm cho attention khó học được vùng cần tập trung ở từng bước giải mã. BiLSTM Context Encoder giúp mỗi vector đặc trưng chứa thông tin từ cả bên trái và bên phải của từ viết tay, từ đó hỗ trợ attention hoạt động ổn định hơn.

4.5.3. Bahdanau Attention

Bahdanau Attention được sử dụng để tính trọng số chú ý giữa trạng thái hiện tại của decoder và các vị trí đặc trưng từ encoder. Tại mỗi bước giải mã t , mô hình tính điểm attention giữa trạng thái ẩn trước đó của decoder s_{t-1} và từng vector encoder h_s :

$$e_{t,s} = v_a^T \tanh(W_a s_{t-1} + U_a h_s)$$

Sau đó, các điểm này được chuẩn hóa bằng softmax:

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{k=1}^T \exp(e_{t,k})}$$

Vector ngữ cảnh tại bước t được tính bằng tổng có trọng số:

$$c_t = \sum_{s=1}^T \alpha_{t,s} h_s$$

Trong đó, $\alpha_{t,s}$ thể hiện mức độ mô hình tập trung vào vị trí s của chuỗi đặc trưng khi sinh ký tự tại bước t .

Cơ chế attention giúp mô hình học căn chỉnh mềm giữa các vùng ảnh và các ký tự đầu ra. Đây là điểm khác biệt quan trọng so với CTC, vốn học căn chỉnh ngầm thông qua hàm mất mát.

4.5.4. GRU Decoder

Decoder của mô hình Attention sử dụng GRUCell. Tại mỗi bước giải mã, decoder nhận đầu vào gồm embedding của ký tự trước đó và vector ngữ cảnh c_t từ attention. Từ đó, decoder cập nhật trạng thái ẩn và dự đoán phân phối xác suất cho ký tự tiếp theo.

Quá trình sinh chuỗi bắt đầu từ token $\langle \text{sos} \rangle$ và kết thúc khi mô hình sinh ra token $\langle \text{eos} \rangle$ hoặc đạt đến độ dài tối đa cho phép.

GRU Decoder là mô hình tự hồi quy, nghĩa là dự đoán tại bước hiện tại phụ thuộc vào các ký tự đã sinh trước đó. Điều này giúp mô hình có khả năng khai thác một phần quan hệ giữa các ký tự trong từ.

4.5.5. Teacher Forcing trong quá trình huấn luyện

Trong quá trình huấn luyện, mô hình Attention sử dụng Teacher Forcing. Thay vì luôn dùng ký tự do mô hình dự đoán ở bước trước, Teacher Forcing cho phép đưa ký tự đúng từ ground truth vào decoder ở một số bước.

Kỹ thuật này giúp quá trình huấn luyện ổn định hơn, đặc biệt trong giai đoạn đầu khi mô hình còn dự đoán sai nhiều. Trong dự án, tỉ lệ Teacher Forcing ban đầu là 0.5 và giảm dần theo epoch. Khi validation, test hoặc predict ảnh thực tế, Teacher Forcing không được sử dụng.

4.6. Giải mã trong mô hình Attention

4.6.1. Greedy Decoding

Greedy Decoding là phương pháp giải mã đơn giản nhất cho mô hình Attention. Tại mỗi bước, decoder chọn ký tự có xác suất cao nhất làm đầu ra. Ký tự này sau đó được đưa vào decoder ở bước tiếp theo.

Phương pháp này có tốc độ nhanh và phù hợp khi cần dự đoán trực tiếp trên ảnh đơn. Tuy nhiên, do chỉ giữ một lựa chọn duy nhất tại mỗi bước, nếu mô hình chọn sai ký tự ở một bước, lỗi đó có thể ảnh hưởng đến các bước sau.

4.6.2. Beam Search

Beam Search là phương pháp giải mã nâng cao hơn. Thay vì chỉ giữ một chuỗi tốt nhất, Beam Search giữ lại nhiều chuỗi ứng viên có xác suất cao nhất tại mỗi bước. Số lượng chuỗi được giữ lại gọi là beam size.

Beam Search giúp mô hình xem xét nhiều khả năng hơn trong quá trình sinh chuỗi, từ đó có thể tạo ra kết quả tốt hơn Greedy Decoding trong một số trường hợp. Tuy nhiên, phương pháp này tốn nhiều thời gian tính toán hơn.

4.7. So sánh hai mô hình

Bảng 4.1 trình bày so sánh tổng quan giữa CTC Baseline và Attention Model.

Bảng 4.1: So sánh kiến trúc CTC Baseline và Attention Model

Tiêu chí	CTC Baseline	Attention Model
CNN Encoder	ResNet18	ResNet18
Sequence Modeling	2-layer BiLSTM	2-layer BiLSTM Context Encoder
Cơ chế căn chỉnh	CTC alignment ngầm	Bahdanau soft attention
Decoder	Không có decoder tự hồi quy	GRU Decoder
Loss Function	CTC Loss	Cross Entropy Loss
Giải mã	CTC Greedy Decoding	Greedy Decoding hoặc Beam Search
Kích thước ảnh đầu vào	64×256	64×512
Độ dài chuỗi đặc trưng	$T = 16$	$T = 32$
Ưu điểm	Đơn giản, ổn định, dễ huấn luyện	Linh hoạt hơn, có thể trực quan hóa attention map
Hạn chế	Bị giới hạn bởi số timestep và giả định độc lập có điều kiện	Phức tạp hơn, huấn luyện chậm hơn, có thể bị lệch attention

4.8. Vai trò của thư mục inference

Trong hệ thống, các thuật toán giải mã được tách riêng trong thư mục `src/inference`. Việc tách riêng phần inference có ý nghĩa quan trọng về mặt thiết kế phần mềm.

Mô hình học sâu không trực tiếp sinh ra chuỗi văn bản hoàn chỉnh theo nghĩa thông thường. Đầu ra của mô

hình thường là tensor chứa logits hoặc phân phối xác suất trên vocabulary. Để chuyển đổi đầu ra này thành chuỗi ký tự, hệ thống cần các thuật toán giải mã riêng.

Các thuật toán như CTC Greedy Decoding, Attention Greedy Decoding và Beam Search đều được đặt trong thư mục `inference`. Cách tổ chức này giúp:

- Tách biệt kiến trúc mô hình và thuật toán giải mã.
- Tránh trùng lặp code giữa các script đánh giá và dự đoán.
- Dễ thay thế hoặc nâng cấp thuật toán giải mã.
- Giúp cấu trúc mã nguồn rõ ràng và dễ bảo trì hơn.

4.9. Tổng kết chương

Chương này đã trình bày chi tiết hai kiến trúc mô hình được sử dụng trong dự án. CTC Baseline là mô hình cơ sở, sử dụng ResNet18, BiLSTM và CTC Loss để nhận dạng chuỗi ký tự mà không cần thông tin căn chỉnh. Attention Model là mô hình chính, sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder để sinh chuỗi ký tự từng bước.

Hai mô hình có cùng mục tiêu nhận dạng từ viết tay nhưng khác nhau về cách căn chỉnh, cách huấn luyện và cách giải mã. Việc triển khai cả hai mô hình giúp nhóm có cơ sở so sánh hiệu quả giữa hướng tiếp cận CTC và hướng tiếp cận Attention trong bài toán word-level HTR.

CHƯƠNG 5. CÀI ĐẶT HỆ THỐNG

5.1. Môi trường phát triển

Dự án được xây dựng bằng ngôn ngữ Python và framework PyTorch. Toàn bộ quá trình huấn luyện, đánh giá và suy diễn được thực hiện trên môi trường local, không sử dụng các dịch vụ OCR API bên thứ ba.

Việc huấn luyện local giúp nhóm chủ động kiểm soát dữ liệu, mô hình, checkpoint và quá trình đánh giá. Đồng thời, điều này cũng giúp dự án thể hiện rõ quy trình xây dựng một hệ thống nhận dạng chữ viết tay từ đầu, thay vì chỉ gọi lại một mô hình hoặc dịch vụ có sẵn.

Bảng 5.1 trình bày môi trường phát triển chính của dự án.

Bảng 5.1: Môi trường phát triển của dự án

Thành phần	Thông tin
Ngôn ngữ lập trình	Python 3.11
Framework học sâu	PyTorch
Xử lý ảnh	OpenCV, Pillow
Xử lý dữ liệu	NumPy, Pandas
Tăng cường dữ liệu	Albumentations
Tính edit distance	editdistance
Trực quan hóa	Matplotlib
Môi trường chạy	Local machine

5.2. Phần cứng sử dụng

Trong quá trình huấn luyện, dự án sử dụng GPU để tăng tốc tính toán. Việc sử dụng GPU là cần thiết vì cả hai mô hình đều có CNN Encoder và các khối xử lý chuỗi, khiến thời gian huấn luyện trên CPU trở nên rất lâu.

Bảng 5.2 trình bày thông tin phần cứng chính được sử dụng.

Bảng 5.2: Phần cứng sử dụng trong quá trình huấn luyện

Thành phần	Thông tin
GPU	NVIDIA GeForce RTX 5060 Ti
VRAM	16 GB
Driver Version	581.80
CUDA Support	CUDA 13.0

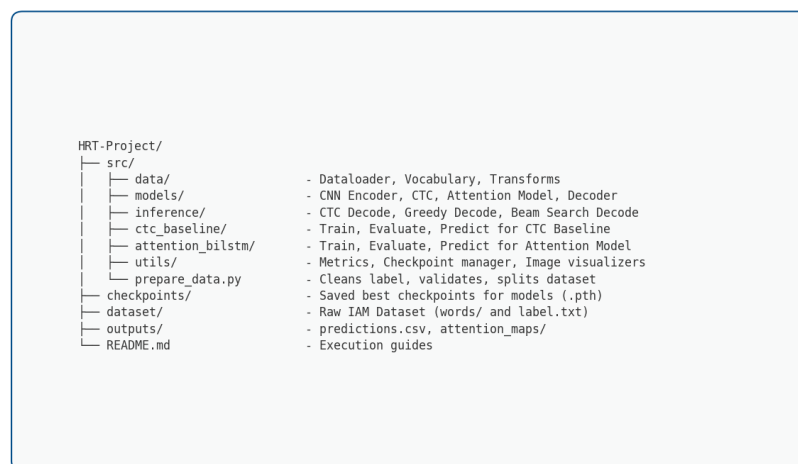
Cấu hình này cho phép huấn luyện và đánh giá mô hình trên tập IAM ở mức word-level với batch size phù hợp, đồng thời lưu checkpoint tốt nhất trong quá trình train.

5.3. Cấu trúc thư mục dự án

Mã nguồn dự án được tổ chức theo hướng tách rõ các thành phần: dữ liệu, mô hình, thuật toán giải mã, tiện ích, pipeline huấn luyện và pipeline suy diễn. Cấu trúc thư mục chính được trình bày dưới đây.

```
1 HRT-Project/  
2     src/  
3         data/  
4         models/  
5             ctc_baseline.py  
6             resnet_encoder.py  
7             attention.py  
8             attention_model.py  
9             gru_decoder.py  
10        inference/  
11            ctc_decode.py  
12            greedy_decode.py  
13            beam_search.py  
14        utils/  
15        ctc_baseline/  
16            train.py  
17            evaluate.py  
18            predict.py  
19        attention_bilstm/  
20            train.py  
21            evaluate.py  
22            predict.py  
23        prepare_data.py  
24        checkpoints/  
25            ctc_baseline/  
26            attention_bilstm/  
27        dataset/  
28        outputs/  
29        requirements.txt  
30        README.md
```

Codebase Directory Structure (Cấu trúc thư mục dự án)



Hình 5.1: Cấu trúc thư mục tổng quát của dự án

5.4. Vai trò của các thư mục chính

5.4.1. Thư mục `src/data`

Thư mục `src/data` chứa các thành phần phục vụ đọc dữ liệu, xử lý nhãn, transform ảnh và xây dựng DataLoader. Đây là phần trung gian giữa dữ liệu thô và mô hình học sâu.

Các nhiệm vụ chính của nhóm file trong thư mục này gồm:

- Đọc danh sách ảnh và nhãn từ file CSV đã xử lý.
- Áp dụng tiền xử lý ảnh.
- Áp dụng tăng cường dữ liệu cho tập train.
- Encode nhãn thành tensor số.
- Padding nhãn trong batch nếu cần.
- Trả về dữ liệu theo định dạng phù hợp với từng mô hình.

5.4.2. Thư mục `src/models`

Thư mục `src/models` chứa các định nghĩa kiến trúc mô hình. Mỗi file đảm nhiệm một thành phần riêng:

- `resnet_encoder.py`: định nghĩa CNN Encoder sử dụng ResNet18.
- `ctc_baseline.py`: định nghĩa mô hình CTC Baseline.
- `attention.py`: định nghĩa Bahdanau Attention.
- `gru_decoder.py`: định nghĩa GRU Decoder.
- `attention_model.py`: kết hợp encoder, attention và decoder thành mô hình hoàn chỉnh.

Cách tách file này giúp mã nguồn dễ đọc hơn. Khi cần chỉnh sửa một thành phần, ví dụ attention hoặc decoder, nhóm có thể chỉnh trực tiếp file tương ứng mà không ảnh hưởng đến toàn bộ hệ thống.

5.4.3. Thư mục `src/inference`

Thư mục `src/inference` chứa các thuật toán giải mã. Đây là phần chuyển đổi đầu ra dạng tensor của mô hình thành chuỗi ký tự có thể đọc được.

Các file chính gồm:

- `ctc_decode.py`: triển khai CTC Greedy Decoding.
- `greedy_decode.py`: triển khai Greedy Decoding cho Attention Model.
- `beam_search.py`: triển khai Beam Search cho Attention Model.

Việc tách riêng inference giúp tránh trùng lặp code giữa `evaluate.py` và `predict.py`. Đồng thời, nó giúp hệ thống dễ mở rộng nếu muốn bổ sung các phương pháp giải mã khác trong tương lai.

5.4.4. Thư mục `src/utils`

Thư mục `src/utils` chứa các hàm tiện ích dùng chung trong toàn dự án. Các chức năng thường gặp gồm:

- Tính metric như Word Accuracy, CER và NED.
- Lưu và load checkpoint.
- Thiết lập seed để tăng tính tái lập.
- Xử lý đường dẫn file.
- Trực quan hóa attention map.

Việc gom các hàm dùng chung vào `utils` giúp giảm trùng lặp code và làm cho các script `train`, `evaluate`, `predict` gọn hơn.

5.4.5. Thư mục `src/ctc_baseline`

Thư mục `src/ctc_baseline` chứa pipeline riêng cho mô hình CTC Baseline. Cụ thể:

- `train.py`: huấn luyện CTC Baseline.
- `evaluate.py`: đánh giá mô hình trên test set.
- `predict.py`: dự đoán ảnh đơn hoặc thư mục ảnh.

5.4.6. Thư mục `src/attention_bilstm`

Thư mục `src/attention_bilstm` chứa pipeline riêng cho mô hình Attention. Cụ thể:

- `train.py`: huấn luyện Attention Model.
- `evaluate.py`: đánh giá mô hình trên test set.
- `predict.py`: dự đoán ảnh đơn hoặc thư mục ảnh, hỗ trợ tùy chọn lưu attention map và Beam Search.

5.5. Pipeline chuẩn bị dữ liệu

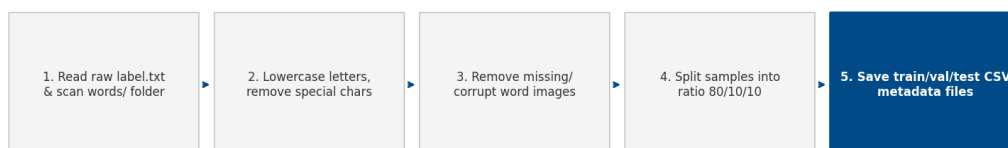
Trước khi huấn luyện, dữ liệu IAM cần được xử lý và chia thành các tập train, validation và test. Quá trình này được thực hiện bằng script:

```
python -m src.prepare_data
```

Script `prepare_data.py` thực hiện các bước chính:

1. Đọc dữ liệu từ thư mục `dataset/`.
2. Đọc file nhãn `label.txt`.
3. Kiểm tra sự tồn tại và tính hợp lệ của ảnh.
4. Làm sạch nhãn.
5. Lọc bỏ các mẫu không hợp lệ.
6. Chia dữ liệu thành train, validation và test.
7. Lưu các file CSV đã xử lý vào thư mục dữ liệu trung gian.

Data Preparation Process Flow (Quy trình chuẩn bị dữ liệu)



Hình 5.2: Luồng chuẩn bị dữ liệu trước khi huấn luyện

Việc tách riêng bước chuẩn bị dữ liệu giúp quá trình huấn luyện tái lập hơn. Sau khi dữ liệu đã được xử lý, các lần huấn luyện sau chỉ cần đọc lại các file CSV thay vì xử lý lại toàn bộ dữ liệu gốc.

5.6. Pipeline huấn luyện

Dự án có hai pipeline huấn luyện riêng cho hai mô hình. Việc tách pipeline giúp cấu hình, loss function và quá trình giải mã của từng mô hình được quản lý rõ ràng.

5.6.1. Huấn luyện CTC Baseline

Mô hình CTC Baseline được huấn luyện bằng lệnh:

```
python -m src.ctc_baseline.train
```

Trong quá trình huấn luyện, ảnh được đưa qua ResNet18 Encoder, reshape thành chuỗi đặc trưng, xử lý bằng BiLSTM và chiếu sang vocabulary. Đầu ra log probability sau đó được đưa vào CTC Loss.

Quy trình huấn luyện CTC Baseline gồm:

1. Load dữ liệu train và validation.
2. Khởi tạo mô hình CTC Baseline.
3. Khởi tạo optimizer và loss function.
4. Lặp qua từng epoch.
5. Tính loss trên tập train.
6. Đánh giá trên tập validation.
7. Lưu checkpoint nếu kết quả validation tốt hơn.

Checkpoint tốt nhất được lưu tại:

```
checkpoints/ctc_baseline/best_ctc_baseline.pth
```

5.6.2. Huấn luyện Attention Model

Mô hình Attention được huấn luyện bằng lệnh:

```
python -m src.attention_bilstm.train
```

Trong quá trình huấn luyện, ảnh được đưa qua ResNet18 Encoder, reshape thành chuỗi đặc trưng và xử lý bởi BiLSTM Context Encoder. Sau đó, decoder sử dụng Bahdanau Attention để sinh chuỗi ký tự từng bước. Loss được tính bằng Cross Entropy Loss, bỏ qua các vị trí padding.

Quy trình huấn luyện Attention Model gồm:

1. Load dữ liệu train và validation.
2. Khởi tạo ResNet18 Encoder, BiLSTM Context Encoder, Attention và GRU Decoder.
3. Encode nhãn với các token đặc biệt như <sos>, <eos> và <pad>.
4. Sử dụng Teacher Forcing trong quá trình train.
5. Tính Cross Entropy Loss tại từng bước decoder.
6. Đánh giá trên validation set với Teacher Forcing bằng 0.
7. Lưu checkpoint tốt nhất.

Checkpoint tốt nhất được lưu tại:

```
checkpoints/attention_bilstm/best_attention_model.pth
```

5.7. Chiến lược lưu checkpoint

Trong quá trình huấn luyện, checkpoint được lưu khi mô hình đạt kết quả tốt hơn trên tập validation. Cách làm này giúp tránh việc sử dụng mô hình ở epoch cuối nếu mô hình đã bắt đầu overfit.

Mỗi checkpoint thường chứa:

- Trọng số mô hình.
- Trạng thái optimizer nếu cần tiếp tục huấn luyện.
- Thông tin epoch.
- Kết quả validation tốt nhất tại thời điểm lưu.
- Cấu hình vocabulary hoặc thông tin cần thiết để decode.

Việc lưu checkpoint giúp quá trình đánh giá và dự đoán ảnh thực tế không cần huấn luyện lại từ đầu.

5.8. Pipeline đánh giá

Sau khi huấn luyện, mô hình được đánh giá trên tập test độc lập. Tập test không được sử dụng trong quá trình cập nhật trọng số hoặc lựa chọn mô hình, do đó kết quả trên tập test phản ánh khách quan hơn khả năng của mô hình.

5.8.1. Đánh giá CTC Baseline

Lệnh đánh giá CTC Baseline:

```
python -m src.ctc_baseline.evaluate --checkpoint checkpoints/ctc_baseline/  
best_ctc_baseline.pth
```

Script đánh giá thực hiện các bước:

1. Load checkpoint đã huấn luyện.
2. Load test set.
3. Chạy suy diễn trên từng batch ảnh.
4. Áp dụng CTC Greedy Decoding.
5. So sánh dự đoán với nhãn thật.
6. Tính Word Accuracy, CER và NED.
7. Lưu dự đoán chi tiết ra file CSV.

5.8.2. Đánh giá Attention Model

Lệnh đánh giá Attention Model:

```
python -m src.attention_bilstm.evaluate --checkpoint checkpoints/  
attention_bilstm/best_attention_model.pth
```

Script đánh giá Attention Model thực hiện các bước tương tự, nhưng sử dụng Greedy Decoding hoặc Beam Search tùy cấu hình. Trong quá trình đánh giá, Teacher Forcing không được sử dụng. Mô hình phải tự sinh chuỗi ký tự đầu ra dựa trên các dự đoán trước đó.

5.9. Pipeline dự đoán ảnh thực tế

Ngoài đánh giá trên test set, hệ thống hỗ trợ dự đoán ảnh đơn lẻ hoặc toàn bộ thư mục ảnh. Chức năng này giúp kiểm thử mô hình với ảnh thực tế bên ngoài dataset.

5.9.1. Dự đoán bằng CTC Baseline

Dự đoán một ảnh đơn:

```
python -m src.ctc_baseline.predict --image duong_dan_anh.png
```

Dự đoán toàn bộ thư mục ảnh:

```
python -m src.ctc_baseline.predict --image duong_dan_thu_muc/
```

Với mô hình CTC, ảnh được tiền xử lý, đưa qua mô hình, sau đó kết quả được giải mã bằng CTC Greedy Decoding.

5.9.2. Dự đoán bằng Attention Model

Dự đoán một ảnh đơn bằng Greedy Decoding:

```
python -m src.attention_bilstm.predict --image duong_dan_anh.png
```

Dự đoán một ảnh đơn và lưu attention map:

```
python -m src.attention_bilstm.predict --image duong_dan_anh.png --  
    save_attention
```

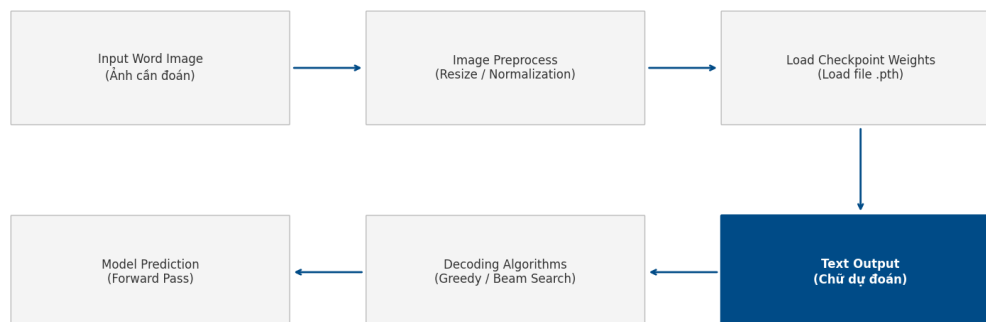
Dự đoán bằng Beam Search:

```
python -m src.attention_bilstm.predict --image duong_dan_anh.png --beam
```

Dự đoán toàn bộ thư mục ảnh:

```
python -m src.attention_bilstm.predict --image duong_dan_thu_muc/
```

Tùy chọn `-save_attention` cho phép lưu attention map vào thư mục `outputs/attention_maps/`. Kết quả này hỗ trợ phân tích vùng ảnh mà mô hình tập trung trong quá trình dự đoán.

Inference Pipeline (Quy trình suy diễn của mô hình)

Hình 5.3: Pipeline suy diễn trên ảnh thực tế

5.10. Đầu ra của hệ thống

Trong quá trình đánh giá và dự đoán, hệ thống tạo ra các đầu ra chính sau:

- Chuỗi ký tự dự đoán cho từng ảnh đầu vào.
- File CSV lưu dự đoán chi tiết trên test set.
- Các chỉ số đánh giá như Word Accuracy, CER và NED.
- Attention map đối với mô hình Attention khi sử dụng tùy chọn `-save_attention`.
- Checkpoint mô hình tốt nhất trong quá trình huấn luyện.

Ví dụ, kết quả dự đoán chi tiết có thể được lưu dưới dạng:

```

1 outputs/predictions_ctc.csv
2 outputs/predictions.csv
3 outputs/attention_maps/
  
```

Các file này giúp nhóm kiểm tra lại từng trường hợp dự đoán đúng, dự đoán sai và phân tích nguyên nhân lỗi của mô hình.

5.11. Quản lý phụ thuộc

Danh sách thư viện cần thiết được lưu trong file:

```

1 requirements.txt
  
```

Người dùng có thể cài đặt các thư viện bằng lệnh:

```

1 python -m pip install -r requirements.txt
  
```

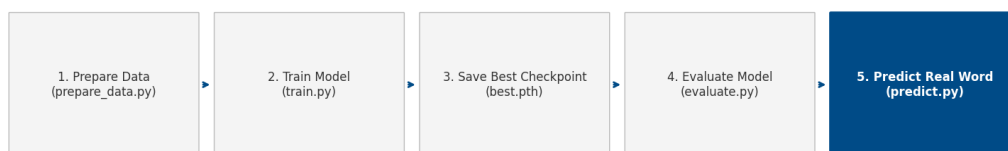

Việc quản lý phụ thuộc bằng `requirements.txt` giúp môi trường chạy dự án dễ tái lập hơn trên máy khác.

5.12. Luồng chạy hệ thống

Luồng chạy đầy đủ của dự án có thể được tóm tắt như sau:

1. Cài đặt môi trường Python và các thư viện cần thiết.
2. Tải IAM Dataset và đặt vào thư mục `dataset/`.
3. Chạy `src.prepare_data` để xử lý dữ liệu.
4. Huấn luyện CTC Baseline hoặc Attention Model.
5. Lưu checkpoint tốt nhất.
6. Đánh giá mô hình trên test set.
7. Dự đoán ảnh thực tế nếu cần.
8. Phân tích kết quả dự đoán và attention map.

End-to-End System Execution Flow (Luồng chạy toàn cục hệ thống)



Hình 5.4: Luồng chạy tổng quát của hệ thống

5.13. Tính mô-đun của hệ thống

Một điểm quan trọng trong cài đặt hệ thống là tính mô-đun. Các thành phần của dự án được tách riêng theo chức năng, giúp dễ phát triển và kiểm thử.

Ví dụ, nếu muốn thay ResNet18 bằng một encoder khác, nhóm có thể chỉnh trong phần `models` mà không cần thay đổi toàn bộ pipeline huấn luyện. Nếu muốn thử một thuật toán giải mã khác, nhóm có thể bổ sung file mới trong `inference`. Nếu muốn thay đổi preprocessing, nhóm có thể chỉnh trong phần `data` hoặc `utils`.

Cách tổ chức này giúp dự án dễ bảo trì hơn, đồng thời thuận lợi cho việc mở rộng trong tương lai.

5.14. Một số lưu ý khi triển khai

Khi chạy dự án trên máy local, cần lưu ý một số điểm:

- Dataset IAM có kích thước lớn, cần đặt đúng cấu trúc thư mục trước khi chạy.
- Checkpoint của mô hình Attention có dung lượng lớn, cần đặt đúng đường dẫn trước khi evaluate hoặc predict.
- Phiên bản PyTorch và CUDA cần tương thích với GPU.
- Ảnh thực tế nên được crop tương đối sát vùng chữ để mô hình dự đoán tốt hơn.
- Với ảnh có nền phức tạp hoặc ánh sáng không đều, kết quả dự đoán có thể kém hơn so với ảnh trong IAM.

5.15. Tổng kết chương

Chương này đã trình bày cách cài đặt hệ thống, bao gồm môi trường phát triển, phần cứng sử dụng, cấu trúc thư mục, vai trò của từng module, pipeline chuẩn bị dữ liệu, huấn luyện, đánh giá và dự đoán ảnh thực tế. Hệ thống được tổ chức theo hướng mô-đun, tách rõ dữ liệu, mô hình, inference và tiện ích, giúp quá trình phát triển, kiểm thử và mở rộng trở nên thuận tiện hơn.

CHƯƠNG 6. THỰC NGHIỆM VÀ KẾT QUẢ

6.1. Mục tiêu thực nghiệm

Mục tiêu của phần thực nghiệm là đánh giá hiệu quả của hai mô hình nhận dạng chữ viết tay mức độ từ đơn lẻ được xây dựng trong dự án: CTC Baseline và Attention Model. Hai mô hình được huấn luyện và đánh giá trên cùng bộ dữ liệu IAM word-level sau tiền xử lý, sử dụng cùng các chỉ số đánh giá để đảm bảo quá trình so sánh có tính nhất quán.

Cụ thể, chương này tập trung trả lời các câu hỏi sau:

- Hai mô hình đạt kết quả như thế nào trên tập kiểm thử IAM?
- Mô hình Attention có cải thiện so với CTC Baseline hay không?
- Sự khác biệt giữa các chỉ số Word Accuracy, CER và NED phản ánh điều gì?
- Mô hình còn gặp những lỗi nhận dạng nào?
- Kết quả trên IAM có phản ánh đầy đủ khả năng nhận dạng ảnh thực tế hay không?

6.2. Thiết lập thực nghiệm

Dữ liệu sử dụng trong thực nghiệm là IAM Handwriting Dataset ở mức word-level. Sau khi lọc bỏ các mẫu không hợp lệ và làm sạch nhãn, tổng số mẫu sử dụng trong dự án là 96,835. Dữ liệu được chia thành ba tập train, validation và test theo tỉ lệ 80/10/10.

Bảng 6.1: Phân chia dữ liệu trong thực nghiệm

Tập dữ liệu	Số mẫu	Tỉ lệ
Train Set	77,468	80%
Validation Set	9,683	10%
Test Set	9,684	10%
Tổng cộng	96,835	100%

Tập train được sử dụng để cập nhật trọng số mô hình. Tập validation được sử dụng để theo dõi quá trình huấn luyện và lựa chọn checkpoint tốt nhất. Tập test được sử dụng ở bước đánh giá cuối cùng.

Hai mô hình được so sánh trong cùng điều kiện dữ liệu:

- **CTC Baseline:** ResNet18 + BiLSTM + CTC Loss.
- **Attention Model:** ResNet18 + BiLSTM Context Encoder + Bahdanau Attention + GRU Decoder.

Bảng 6.2: Tóm tắt thiết lập thực nghiệm

Thành phần	Thiết lập
Bộ dữ liệu	IAM Handwriting Dataset word-level
Tổng số mẫu sau tiền xử lý	96,835
Tập kiểm thử	9,684 mẫu
Mô hình baseline	ResNet18 + BiLSTM + CTC Loss
Mô hình attention	ResNet18 + BiLSTM + Bahdanau Attention + GRU Decoder
Metric	Word Accuracy, CER, NED
Môi trường huấn luyện	Local machine sử dụng PyTorch

6.3. Các chỉ số đánh giá

Dự án sử dụng ba chỉ số đánh giá chính: Word Accuracy, Character Error Rate và Normalized Edit Distance.

6.3.1. Word Accuracy

Word Accuracy đo tỷ lệ số từ được dự đoán chính xác hoàn toàn:

$$WA = \frac{\text{Số từ dự đoán đúng hoàn toàn}}{\text{Tổng số từ kiểm thử}} \times 100\%$$

Đây là chỉ số nghiêm ngặt vì chỉ cần mô hình dự đoán sai một ký tự thì cả từ được xem là sai.

6.3.2. Character Error Rate

Character Error Rate, viết tắt là CER, đo tỷ lệ lỗi ở mức ký tự:

$$CER = \frac{\sum \text{edit_distance}(\text{pred}, \text{target})}{\sum \text{len}(\text{target})} \times 100\%$$

CER giúp đánh giá mức độ sai khác chi tiết giữa chuỗi dự đoán và chuỗi nhãn đúng. Trong một số trường hợp, một từ bị tính là sai theo Word Accuracy nhưng thực tế chỉ sai một ký tự; CER phản ánh được mức độ sai này tốt hơn.

6.3.3. Normalized Edit Distance

Normalized Edit Distance, viết tắt là NED, đo mức độ tương đồng giữa chuỗi dự đoán và chuỗi nhãn sau khi chuẩn hóa theo độ dài chuỗi:

$$NED = 1 - \frac{1}{N} \sum_{i=1}^N \frac{\text{edit_distance}(\text{pred}_i, \text{target}_i)}{\max(\text{len}(\text{pred}_i), \text{len}(\text{target}_i))}$$

NED càng cao thì dự đoán càng gần với nhãn đúng.

6.4. Kết quả trên tập kiểm thử IAM

Kết quả đánh giá trên 9,684 mẫu kiểm thử được trình bày trong Bảng 6.3.

Bảng 6.3: Kết quả so sánh hai mô hình trên IAM Test Set

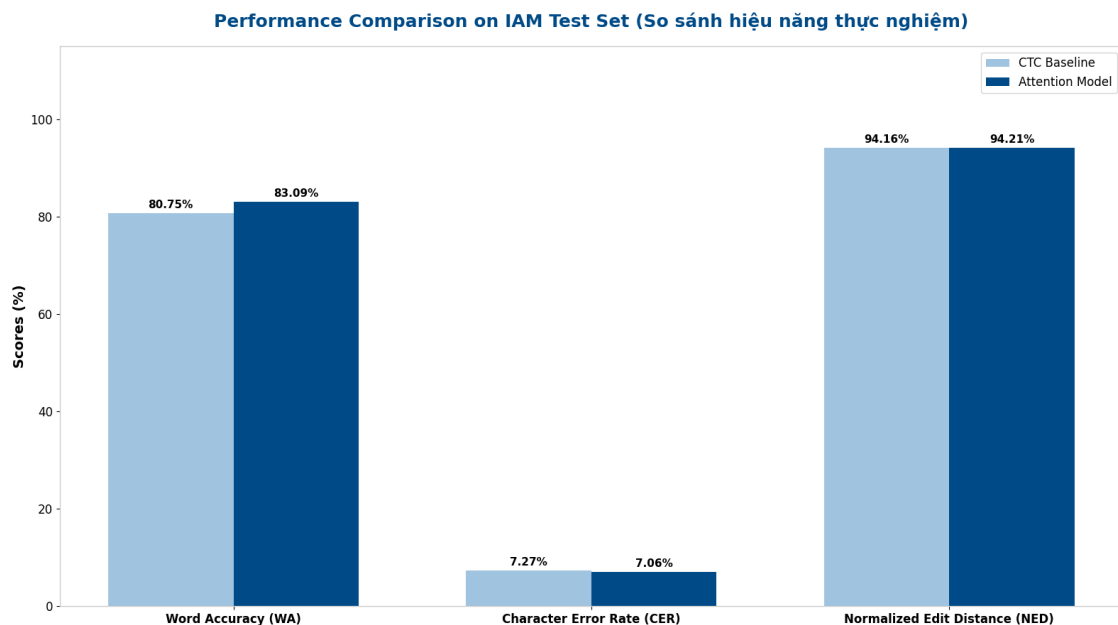
Mô hình	Kiến trúc	Word Accuracy	CER
CTC Baseline	ResNet18 + BiLSTM + CTC Loss	80.75%	7.27%
Attention Model	ResNet18 + BiLSTM + Bahdanau Attention + GRU Decoder	83.09%	7.06%

Bảng 6.4 trình bày thêm chỉ số NED và dung lượng checkpoint.

Bảng 6.4: Kết quả chi tiết của hai mô hình

Chỉ số	CTC Baseline	Attention Model
Word Accuracy	80.75%	83.09%
Character Error Rate	7.27%	7.06%
Normalized Edit Distance	94.16%	94.21%
Dung lượng checkpoint	khoảng 75 MB	khoảng 91 MB

Có thể thấy Attention Model đạt Word Accuracy cao hơn CTC Baseline. Cụ thể, Attention Model đạt 83.09%, trong khi CTC Baseline đạt 80.75%. Chênh lệch tuyệt đối là 2.34 điểm phần trăm. Về CER, Attention Model đạt 7.06%, thấp hơn 7.27% của CTC Baseline.



Hình 6.1: Biểu đồ so sánh kết quả CTC Baseline và Attention Model

6.5. Phân tích kết quả

6.5.1. So sánh Word Accuracy

Word Accuracy của Attention Model cao hơn CTC Baseline cho thấy mô hình Attention dự đoán đúng hoàn toàn nhiều từ hơn trên tập kiểm thử. Điều này có thể đến từ khả năng học căn chỉnh mềm giữa đặc trưng ảnh và chuỗi ký tự đầu ra.

Trong mô hình CTC, đầu ra được dự đoán theo từng timestep và sau đó rút gọn bằng quy tắc CTC Decoding. Cách tiếp cận này đơn giản và ổn định, nhưng không sử dụng decoder tự hồi quy. Ngược lại, Attention Model

sinh từng ký tự dựa trên trạng thái trước đó, vector ngữ cảnh và ký tự đã sinh ở bước trước. Do đó, mô hình có thêm khả năng khai thác quan hệ giữa các ký tự trong từ.

Tuy nhiên, mức cải thiện Word Accuracy không quá lớn. Điều này cho thấy CTC Baseline vẫn là một mô hình có hiệu quả tốt đối với bài toán word-level HTR trên IAM Dataset.

6.5.2. So sánh Character Error Rate

CER của Attention Model thấp hơn CTC Baseline, nhưng mức chênh lệch không lớn. Điều này cho thấy hai mô hình có mức độ sai khác ở cấp độ ký tự tương đối gần nhau. Một phần các lỗi dự đoán có thể chỉ là sai một vài ký tự trong từ, thay vì sai hoàn toàn toàn bộ chuỗi.

Việc CER của Attention thấp hơn cho thấy mô hình này có xu hướng dự đoán chuỗi gần với nhãn đúng hơn. Tuy vậy, vì CER chỉ giảm từ 7.27% xuống 7.06%, chưa thể kết luận rằng mô hình Attention vượt trội tuyệt đối trong mọi trường hợp. Kết quả này nên được hiểu là mô hình Attention có cải thiện nhẹ và ổn định hơn trên thiết lập thực nghiệm hiện tại.

6.5.3. So sánh Normalized Edit Distance

NED của hai mô hình khá gần nhau: CTC Baseline đạt 94.16%, còn Attention Model đạt 94.21%. Chênh lệch này rất nhỏ, cho thấy xét theo khoảng cách chỉnh sửa đã chuẩn hóa, hai mô hình tạo ra chuỗi dự đoán có mức độ tương đồng với nhãn thật tương đối gần nhau.

Điều này cũng phản ánh rằng sự khác biệt giữa hai mô hình rõ hơn ở Word Accuracy so với NED. Attention Model có thể giúp tăng số lượng từ dự đoán đúng hoàn toàn, nhưng xét trung bình khoảng cách chỉnh sửa thì mức cải thiện không quá lớn.

6.6. Vai trò của BiLSTM Context Encoder trong mô hình Attention

Một điểm quan trọng trong Attention Model là việc sử dụng BiLSTM Context Encoder trước cơ chế attention. Nếu chỉ sử dụng đặc trưng cục bộ từ CNN, mô hình có thể gặp khó khăn khi xác định vùng ảnh tương ứng với từng ký tự, đặc biệt trong các trường hợp chữ viết liền nét hoặc ký tự có hình dạng gần giống nhau.

BiLSTM Context Encoder giúp mỗi vị trí đặc trưng chứa thêm thông tin từ cả hai phía của từ viết tay. Nhờ đó, attention có cơ sở tốt hơn để tính trọng số chú ý tại từng bước giải mã. Việc bổ sung khối ngữ cảnh này giúp mô hình Attention hội tụ ổn định hơn và đạt kết quả tốt hơn so với baseline trong thực nghiệm.

6.7. Một số ví dụ dự đoán

Ngoài các chỉ số định lượng, việc quan sát trực tiếp các ví dụ dự đoán giúp hiểu rõ hơn hành vi của mô hình. Bảng 6.5 là mẫu bảng có thể dùng để trình bày một số trường hợp dự đoán đúng và sai.

Bảng 6.5: Một số ví dụ dự đoán của hai mô hình

Ảnh	Nhãn đúng	CTC dự đoán	Attention dự đoán	Nhận xét
Hình 1	example	example	example	Cả hai mô hình dự đoán đúng
Hình 2	writing	writng	writing	Attention dự đoán đầy đủ hơn
Hình 3	certain	cetrain	certain	CTC nhầm thứ tự một số ký tự
Hình 4	different	differnt	differnt	Cả hai mô hình còn thiếu ký tự

Trong báo cáo hoàn chỉnh, nhóm có thể thay các dòng minh họa trong bảng bằng kết quả thật lấy từ file dự đoán `outputs/predictions.csv` và `outputs/predictions_ctc.csv`.

Model Prediction Comparison Examples (Mẫu kết quả nhận dạng)

Image (Ảnh)	Ground Truth	CTC Prediction	Attention Prediction	Result
sample 1	example	example	example	Both Correct
sample 2	writing	writng	writing	Attention Correct
sample 3	certain	cetrain	certain	Attention Correct
sample 4	different	differnt	differnt	Both Incorrect

Hình 6.2: Ví dụ trực quan một số dự đoán đúng và sai

6.8. Trực quan hóa Attention

Một ưu điểm của mô hình Attention là có thể trực quan hóa trọng số attention tại từng bước giải mã. Attention map cho biết mô hình đang tập trung vào vùng nào của ảnh khi dự đoán một ký tự cụ thể.

Khi chạy dự đoán với tùy chọn `-save_attention`, hệ thống lưu attention map vào thư mục `outputs/attention_maps/`. Các bản đồ này giúp quan sát quá trình mô hình căn chỉnh giữa vùng ảnh và chuỗi ký tự đầu ra.

Bahdanau Attention Map Visualization Example (Minh họa bản đồ chú ý)



Hình 6.3: Ví dụ attention map của mô hình Attention

Attention map có ý nghĩa hỗ trợ giải thích, nhưng không nên xem đây là bằng chứng tuyệt đối rằng mô hình luôn hiểu đúng cấu trúc chữ viết. Trong một số trường hợp, attention có thể tập trung lệch sang vùng trắng, vùng padding hoặc vùng nhiễu. Điều này đặc biệt dễ xảy ra khi ảnh đầu vào khác nhiều so với phân phối của IAM Dataset.

6.9. Kiểm thử trên ảnh thực tế

Bên cạnh đánh giá trên IAM Test Set, nhóm cũng thử nghiệm mô hình trên một số ảnh chữ viết tay thực tế. Đây là bước kiểm tra bổ sung nhằm quan sát khả năng hoạt động của mô hình ngoài dữ liệu chuẩn.

Kết quả thực tế cho thấy mô hình có thể dự đoán tốt hơn với các ảnh có đặc điểm gần với IAM, ví dụ chữ viết rõ, nền sạch, ảnh được crop sát vùng chữ và độ nghiêng không quá lớn. Tuy nhiên, với chữ viết cá nhân, ảnh chụp bằng điện thoại, nền không đồng đều, ánh sáng kém hoặc nét chữ khác nhiều so với IAM, kết quả có thể giảm đáng kể.

Real-World Test Examples Placeholder (Thử nghiệm ảnh thực tế)



Hình 6.4: Một số ví dụ kiểm thử trên ảnh chữ viết tay thực tế

Hiện tượng này phản ánh vấn đề domain shift, tức là dữ liệu thực tế có phân phối khác với dữ liệu huấn luyện. Vì vậy, kết quả tốt trên IAM Test Set chưa đủ để khẳng định mô hình có thể hoạt động tốt trong mọi bối cảnh thực tế.

6.10. Phân tích lỗi

Qua quan sát các trường hợp dự đoán sai, có thể chia lỗi của mô hình thành một số nhóm chính:

- **Nhầm ký tự có hình dạng gần giống nhau:** ví dụ o và a, r và v, l và i.
- **Thiếu hoặc thừa ký tự:** mô hình có thể bỏ sót ký tự trong từ dài hoặc sinh thêm ký tự không có trong nhãn.
- **Sai ở vùng chữ viết không rõ:** các nét chữ mờ, dính hoặc nghiêng mạnh dễ gây nhầm lẫn.
- **Ảnh ngoài phân phối IAM:** ảnh thực tế có nền, ánh sáng, độ dày nét hoặc kiểu chữ khác biệt có thể làm giảm chất lượng dự đoán.
- **Attention lệch vùng:** trong một số trường hợp, attention có thể tập trung vào vùng padding hoặc vùng trắng thay vì nét chữ.

Những lỗi này cho thấy bài toán nhận dạng chữ viết tay vẫn phụ thuộc nhiều vào chất lượng dữ liệu, cách tiền xử lý và mức độ đa dạng của tập huấn luyện.

6.11. Đánh giá chung

Từ các kết quả thực nghiệm, có thể rút ra một số nhận xét:

- Cả hai mô hình đều học được đặc trưng chữ viết tay trên IAM Dataset và đạt kết quả tương đối tốt trên tập kiểm thử.

- Attention Model đạt Word Accuracy cao hơn CTC Baseline và CER thấp hơn một chút.
- Mức cải thiện của Attention Model là có nhưng không quá lớn, đặc biệt khi xét theo NED.
- CTC Baseline vẫn là một mô hình đơn giản, ổn định và có hiệu quả tốt.
- Attention Model có ưu điểm là linh hoạt hơn và có thể trực quan hóa attention map, nhưng phức tạp hơn và dễ bị ảnh hưởng bởi chất lượng căn chỉnh.
- Kết quả trên IAM Test Set chưa phản ánh đầy đủ khả năng hoạt động trên ảnh chữ viết tay thực tế.

6.12. Tổng kết chương

Chương này đã trình bày thiết lập thực nghiệm, các chỉ số đánh giá, kết quả so sánh giữa CTC Baseline và Attention Model, cũng như phân tích một số nhóm lỗi thường gặp. Kết quả cho thấy Attention Model có cải thiện so với CTC Baseline trên IAM Test Set, đặc biệt ở Word Accuracy. Tuy nhiên, mức cải thiện không quá lớn và khả năng tổng quát hóa sang ảnh thực tế vẫn còn là vấn đề cần được tiếp tục xem xét.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

Trong phạm vi bài tập lớn, nhóm đã xây dựng một hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ sử dụng PyTorch. Hệ thống nhận đầu vào là ảnh chứa một từ tiếng Anh viết tay và sinh ra chuỗi ký tự tương ứng. Dự án được triển khai hoàn toàn trên môi trường local, không sử dụng các dịch vụ OCR API bên thứ ba.

Nhóm đã thực hiện đầy đủ các bước chính của một pipeline học sâu cho bài toán nhận dạng chữ viết tay, bao gồm chuẩn bị dữ liệu, tiền xử lý ảnh, xây dựng vocabulary, thiết kế mô hình, huấn luyện, đánh giá và dự đoán ảnh thực tế. Bộ dữ liệu sử dụng trong dự án là IAM Handwriting Dataset ở mức word-level sau khi đã được làm sạch và chia thành các tập train, validation và test.

Về mặt mô hình, nhóm đã triển khai và so sánh hai kiến trúc. Mô hình thứ nhất là CTC Baseline, kết hợp ResNet18, BiLSTM và CTC Loss. Mô hình thứ hai là Attention Model, kết hợp ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder.

Kết quả thực nghiệm trên tập kiểm thử IAM cho thấy CTC Baseline đạt 80.75% Word Accuracy và 7.27% CER. Attention Model đạt 83.09% Word Accuracy và 7.06% CER. Trên thiết lập thực nghiệm này, Attention Model cho kết quả tốt hơn CTC Baseline, đặc biệt ở chỉ số Word Accuracy. Tuy nhiên, mức cải thiện không quá lớn và cần được nhìn nhận trong phạm vi bộ dữ liệu cũng như điều kiện thực nghiệm hiện tại.

Dự án giúp nhóm hiểu rõ hơn về sự khác biệt giữa hai hướng tiếp cận phổ biến trong bài toán nhận dạng chuỗi từ ảnh. CTC có ưu điểm là đơn giản, ổn định và không cần decoder tự hồi quy. Trong khi đó, Attention Model linh hoạt hơn, có khả năng học căn chỉnh mềm giữa vùng ảnh và ký tự đầu ra, đồng thời hỗ trợ trực quan hóa attention map. Mỗi hướng tiếp cận đều có ưu điểm và hạn chế riêng.

7.2. Những kết quả đạt được

Trong quá trình thực hiện, nhóm đã đạt được một số kết quả chính sau:

- Xây dựng được pipeline chuẩn bị dữ liệu cho IAM Handwriting Dataset ở mức word-level.
- Cài đặt được mô hình CTC Baseline sử dụng ResNet18, BiLSTM và CTC Loss.
- Cài đặt được mô hình Attention sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder.
- Xây dựng được các script phục vụ huấn luyện, đánh giá và dự đoán ảnh đơn hoặc thư mục ảnh.
- Tích hợp các phương pháp giải mã gồm CTC Greedy Decoding, Attention Greedy Decoding và Beam Search.
- Đánh giá hai mô hình bằng các chỉ số Word Accuracy, CER và NED.

- Trực quan hóa được attention map đối với mô hình Attention để hỗ trợ phân tích kết quả.
- Tổ chức mã nguồn theo cấu trúc tương đối rõ ràng, tách riêng dữ liệu, mô hình, inference, utils và pipeline của từng mô hình.

Các kết quả trên cho thấy hệ thống đã đáp ứng được các mục tiêu cơ bản đặt ra ban đầu của bài tập lớn. Tuy vậy, hệ thống vẫn chủ yếu dừng ở mức nghiên cứu và thực nghiệm, chưa phải một sản phẩm OCR hoàn chỉnh có thể triển khai ổn định trong môi trường thực tế.

7.3. Hạn chế của dự án

Mặc dù hệ thống đạt được kết quả tương đối khả quan trên IAM Test Set, dự án vẫn còn một số hạn chế.

Thứ nhất, hệ thống mới xử lý bài toán nhận dạng ở mức từ đơn lẻ. Mỗi ảnh đầu vào được giả định chỉ chứa một từ tiếng Anh. Dự án chưa xử lý các bài toán phức tạp hơn như nhận dạng dòng văn bản, đoạn văn hoặc toàn bộ tài liệu. Các bước như phát hiện vùng chữ, tách dòng, tách từ và xử lý bố cục tài liệu chưa nằm trong phạm vi triển khai.

Thứ hai, mô hình phụ thuộc nhiều vào phân phối của IAM Dataset. Ảnh trong IAM thường đã được chuẩn hóa và cắt tương đối sát vùng chữ. Khi chuyển sang ảnh chữ viết tay thực tế, đặc biệt là ảnh chụp bằng điện thoại, nền không đồng đều, ánh sáng kém hoặc nét chữ khác nhiều, kết quả dự đoán có thể giảm đáng kể.

Thứ ba, khả năng tổng quát hóa của mô hình vẫn còn hạn chế. Kết quả tốt trên tập test của IAM không đồng nghĩa với việc mô hình có thể nhận dạng tốt mọi kiểu chữ viết tay ngoài thực tế. Đây là vấn đề thường gặp trong các mô hình học sâu khi dữ liệu triển khai khác với dữ liệu huấn luyện.

Thứ tư, mô hình Attention vẫn có thể gặp hiện tượng attention lệch sang vùng padding hoặc vùng trắng ở cuối ảnh. Nguyên nhân có thể đến từ việc chưa áp dụng attention padding mask đầy đủ trong quá trình tính trọng số attention.

Thứ năm, Beam Search trong dự án mới dừng ở mức giải mã dựa trên xác suất của mô hình, chưa kết hợp với Language Model ngoài. Vì vậy, mô hình chưa tận dụng được tri thức ngôn ngữ ở mức từ vựng hoặc ngữ cảnh rộng hơn.

Thứ sáu, dự án chưa tập trung tối ưu tốc độ suy luận, dung lượng mô hình hoặc khả năng đóng gói thành ứng dụng có giao diện người dùng. Các script hiện tại phù hợp cho thực nghiệm và kiểm thử hơn là triển khai thực tế.

7.4. Bài học rút ra

Thông qua quá trình thực hiện dự án, nhóm rút ra một số bài học.

Trước hết, đối với các bài toán học sâu, chất lượng dữ liệu và tiền xử lý ảnh có ảnh hưởng rất lớn đến kết quả. Ngay cả khi mô hình có kiến trúc tương đối tốt, ảnh đầu vào không phù hợp hoặc khác phân phối huấn luyện vẫn có thể làm kết quả suy giảm.

Bên cạnh đó, việc lựa chọn chỉ số đánh giá cũng rất quan trọng. Word Accuracy phản ánh khả năng dự đoán đúng hoàn toàn cả từ, trong khi CER cho cái nhìn chi tiết hơn ở cấp độ ký tự. Nếu chỉ nhìn vào một chỉ số, việc đánh giá mô hình có thể chưa đầy đủ.

Ngoài ra, việc so sánh baseline với mô hình cải tiến là cần thiết. CTC Baseline tuy đơn giản hơn nhưng vẫn đạt kết quả tốt, giúp nhóm có mốc tham chiếu rõ ràng khi đánh giá mô hình Attention. Điều này tránh việc chỉ nhìn vào một mô hình đơn lẻ mà không biết kết quả đó tốt hay chưa trong cùng điều kiện thực nghiệm.

Cuối cùng, kết quả thực nghiệm cần được diễn giải thận trọng. Mô hình có thể đạt kết quả tốt trên bộ dữ liệu chuẩn nhưng vẫn gặp khó khăn khi đưa ra ngoài thực tế. Do đó, khi đánh giá một hệ thống nhận dạng chữ viết tay, cần xét cả kết quả định lượng trên test set và kết quả kiểm thử bổ sung trên ảnh thực tế.

7.5. Hướng phát triển

Nếu tiếp tục phát triển, dự án có thể được cải thiện theo một số hướng sau.

7.5.1. Cải thiện tiền xử lý ảnh thực tế

Một hướng phát triển quan trọng là cải thiện pipeline tiền xử lý cho ảnh ngoài thực tế. Ảnh chụp bằng điện thoại thường có nền phức tạp, ánh sáng không đều, góc chụp lệch hoặc vùng chữ không được crop sát. Việc bổ sung các bước như làm sạch nền, cân bằng sáng, deskew, crop tự động và chuẩn hóa độ dày nét có thể giúp mô hình hoạt động ổn định hơn trên ảnh thực tế.

7.5.2. Bổ sung dữ liệu đa dạng hơn

Để cải thiện khả năng tổng quát hóa, có thể bổ sung thêm dữ liệu chữ viết tay từ nhiều người viết khác nhau, nhiều điều kiện chụp khác nhau và nhiều kiểu nét chữ khác nhau. Việc fine-tune mô hình trên dữ liệu thực tế đa dạng có thể giúp giảm khoảng cách giữa IAM Dataset và môi trường sử dụng thật.

7.5.3. Áp dụng Attention Padding Mask

Đối với mô hình Attention, có thể bổ sung attention padding mask để hạn chế mô hình tập trung vào vùng padding trắng. Điều này có thể giúp attention map ổn định hơn, đặc biệt với các ảnh có từ ngắn và nhiều khoảng trắng ở phía sau.

7.5.4. Kết hợp Language Model

Một hướng cải thiện khác là kết hợp mô hình nhận dạng với Language Model. Language Model có thể hỗ trợ lựa chọn chuỗi đầu ra hợp lý hơn, đặc biệt trong các trường hợp mô hình thị giác phân vân giữa nhiều ký tự có hình dạng gần giống nhau. Tuy nhiên, hướng này cần được thiết kế cẩn thận để tránh làm sai lệch kết quả đối với các từ ít gặp hoặc tên riêng.

7.5.5. Thử nghiệm các kiến trúc mới

Trong tương lai, có thể thử nghiệm các kiến trúc mạnh hơn như Transformer-based HTR, CRNN cải tiến hoặc mô hình kết hợp CTC–Attention. Các kiến trúc này có thể giúp mô hình học quan hệ dài hạn tốt hơn hoặc cải thiện quá trình căn chỉnh giữa ảnh và chuỗi ký tự.

7.5.6. Mở rộng sang line-level recognition

Dự án hiện chỉ xử lý word-level recognition. Một hướng phát triển tự nhiên là mở rộng sang nhận dạng dòng văn bản. Tuy nhiên, line-level HTR khó hơn vì chuỗi đầu ra dài hơn, ảnh có nhiều từ hơn và yêu cầu mô hình xử lý khoảng trắng cũng như cấu trúc dòng tốt hơn.

7.5.7. Xây dựng ứng dụng demo

Nếu có thêm thời gian, hệ thống có thể được đóng gói thành một ứng dụng demo đơn giản. Người dùng có thể tải ảnh lên, chọn mô hình CTC hoặc Attention, xem kết quả dự đoán và quan sát attention map. Ứng dụng demo sẽ giúp việc trình bày và kiểm thử hệ thống trực quan hơn.

7.6. Kết luận chung

Nhìn chung, dự án đã hoàn thành được mục tiêu chính là xây dựng và so sánh hai mô hình nhận dạng chữ viết tay mức độ từ đơn lẻ trên bộ dữ liệu IAM. Kết quả cho thấy mô hình Attention có cải thiện so với CTC Baseline trong thiết lập thực nghiệm của nhóm, nhưng sự cải thiện này cần được nhìn nhận trong phạm vi dữ liệu và điều kiện đánh giá hiện tại.

Hệ thống hiện tại phù hợp với mục tiêu học tập, thực nghiệm và so sánh mô hình trong khuôn khổ bài tập lớn. Tuy nhiên, để có thể sử dụng ổn định trong thực tế, mô hình cần được cải thiện thêm về dữ liệu, tiền xử lý, khả năng tổng quát hóa và cách triển khai. Đây cũng là những hướng mà nhóm có thể tiếp tục nghiên cứu nếu phát triển dự án ở các giai đoạn sau.

TÀI LIỆU THAM KHẢO

- [1] U. V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *International Journal on Document Analysis and Recognition*, vol. 5, pp. 39–46, 2002.
- [2] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” in *Proceedings of the 23rd International Conference on Machine Learning*, pp. 369–376, 2006.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *International Conference on Learning Representations*, 2015.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [5] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp. 1724–1734, 2014.
- [7] B. Shi, X. Bai, and C. Yao, “An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.
- [8] PyTorch Contributors, “PyTorch Documentation,” Available: <https://pytorch.org/docs/stable/index.html>.
- [9] PyTorch Contributors, “Torchvision Models and Pre-trained Weights,” Available: <https://pytorch.org/vision/stable/models.html>.
- [10] Albumentations Team, “Albumentations Documentation,” Available: <https://albumentations.ai/docs/>.
- [11] OpenCV Team, “OpenCV Documentation,” Available: <https://docs.opencv.org/>.
- [12] IAM Handwriting Database, “IAM Handwriting Database Official Website,” Available: <https://fki.tic.heia-fr.ch/databases/iam-handwriting-database>.